Державний торговельно-економічний університет

Кафедра інженерії програмного забезпечення та кібербезпеки

КВАЛІФІКАЦІЙНА РОБОТА

на тему:

«Програмне рішення для автентифікації та авторизації користувачів на платформі .NET»

Студента 4 курсу, 5 групи, спеціальності «Інженерія програмного забезпечення» освітньої програми «Інженерія програмного забезпечення»

_____
підпис студента

Тютюн Анастасії Вікторівни

Науковий керівник к.е.н., старший викладач кафедри інженерії програмного забезпечення та кібербезпеки

_____
підпис керівника

Бульба Сергій Андрійович

Гарант освітньої програми к.е.н., старший викладач кафедри інженерії програмного забезпечення та кібербезпеки

_____
підпис гаранта

Бульба Сергій Андрійович

КИЇВ – 2026

Державний торговельно-економічний університет

Факультет інформаційних технологій
Кафедра інженерії програмного забезпечення та кібербезпеки
Ступінь вищої освіти «Бакалавр»
Спеціальність «Інженерія програмного забезпечення»
Освітньо-професійна програма «Інженерія програмного забезпечення»

Затверджую

Зав. кафедри інженерії програмного забезпечення та кібербезпеки
_ Десятко А.М.
«___» грудня____2025р.

Завдання

на кваліфікаційну роботу здобувачу вищої освіти

Тютюн Анастасії Вікторівні

(прізвище, ім'я, по батькові)

1. Тема кваліфікаційної роботи

«Програмне рішення для автентифікації та авторизації користувачів на платформі .NET»

Затверджена наказом ДТЕУ від «__» грудня 2025р. №_____

2. Строк здачі здобувачем закінченої роботи «___» червня 2026 р.

3. Цільова установка та вихідні дані до роботи

Мета роботи: створення програмного рішення для автентифікації та авторизації користувачів у розподілених системах засобами платформи .NET із використанням протоколу OpenID Connect.

Об'єкт дослідження: процеси підтвердження особи користувача та керування доступом до ресурсів у сучасних інформаційних системах, засновані на актуальних стандартах і протоколах.

Предмет дослідження: аналіз та реалізація механізмів автентифікації та авторизації між кількома вебдодатками на основі протоколу OpenID Connect у

середовищі .NET.

4. Консультанти по роботі із зазначенням розділів, за якими здійснюється консультування:

Розділ	Консультант (прізвище, ініціали)	Підпис, дата	
		Завдання видано	Завдання виконано

5. Зміст кваліфікаційної роботи (перелік питань за кожним розділом)

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ

ВСТУП

РОЗДІЛ 1. Дослідження предметної області та підходів до реалізації програмного рішення

1.1. Аналіз предметної області

1.2. Визначення проблеми та можливих шляхів її вирішення

1.3. Характеристика потенційних користувачів

1.4. Технічне завдання на створення програмного рішення

Висновки до розділу 1

РОЗДІЛ 2. Проєктування вимог до розробки програмного рішення

2.1. Теоретичні засади реалізації механізмів автентифікації та керування доступом

2.2. Проєктування функціональних вимог

2.2.1. Функціональні характеристики рішення

2.2.2. Визначення сценаріїв використання

2.2.3. Розробка користувацького і програмного інтерфейсів у рішенні

2.3. Проєктування нефункціональних вимог

2.3.1. Характеристика основних нефункціональних вимог до рішення

2.3.2. Діаграма класів (class diagram)

2.3.3. Діаграма послідовності (sequence diagram)

2.3.4. Діаграма діяльності (activity diagram)

2.3.5. Діаграма компонентів (component diagram)
2.3.6. Діаграма розгортання (deployment diagram)
Висновки до розділу 2
РОЗДІЛ 3. Розробка програмного рішення
3.1. Середовище розробки та стек використаних технологій
3.2. Опис архітектури та реалізації програмного рішення
3.3. Демонстрація роботи програмного рішення
3.4. Демонстрація роботи програмного рішення
Висновки до розділу 3
ВИСНОВКИ
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ
ДОДАТКИ

6. Календарний план виконання робіт

№ пор.	Назва етапів кваліфікаційної роботи	Строк виконання етапів роботи	
		за планом	фактично
1	2	3	4
1	Вибір теми кваліфікаційної роботи	31.10.2025	31.10.2025
2	Розробка та затвердження завдання кваліфікаційної роботи	24.11.2025	24.11.2025
3	Вступ та перелік літературних джерел	12.12.2025	12.12.2025
4	РОЗДІЛ 1	16.01.2026	16.01.2026
5	РОЗДІЛ 2	20.02.2026	20.02.2026
6	РОЗДІЛ 3	20.03.2026	20.03.2026
7	Висновки	20.03.2026	20.03.2026
8	Здача кваліфікаційної роботи на кафедру (перша перевірка)	27.03.2026	27.03.2026
9	Підготовка реферату та презентації доповіді	03.04.2026	03.04.2026
10	Попередній захист кваліфікаційної роботи	06.04.2026 - 17.04.2026	06.04.2026 - 17.04.2026
11	Зовнішнє рецензування кваліфікаційної роботи	20.04.2026 - 24.04.2026	20.04.2026 - 24.04.2026
12	Здача зброшурованої кваліфікаційної роботи	25.04.2026 - 01.05.2026	25.04.2026 - 01.05.2026
13	Публічний захист кваліфікаційної роботи	16.06.2025- 20.06.2025	

7. Дата видачі завдання «___» грудня 2025 р.

8. Науковий керівник кваліфікаційної роботи

_____

(прізвище, ініціали, підпис)

9. Гарант освітньої програми _____

(прізвище, ініціали, підпис)

10. Завдання прийняв до виконання студент _____

(прізвище, ініціали, підпис)

11. Відгук керівника кваліфікаційної роботи

_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____
_____

Науковий керівник кваліфікаційної роботи _____
(підпис, дата)

Відмітка про попередній захист _____
(ПІБ, підпис, дата)

12. Висновок про кваліфікаційну роботу

Кваліфікаційна робота студента _____
(прізвище, ініціали)

може бути допущена до захисту екзаменаційній комісії.

Гарант освітньої програми _____
(прізвище, ініціали, підпис)

Завідувач кафедри _____
(підпис, прізвище, ініціали)

«____»_____ 20___ р.

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ

1. API – Application Programming Interface (прикладний програмний інтерфейс)
2. HTTP(S) – Hypertext Transfer Protocol (Secure) (протокол передачі гіпертексту (захищений))
3. IaaS – Infrastructure-as-a-Service (інфраструктура як сервіс)
4. IAM – Identity and Access Management (управління ідентифікацією та доступом)
5. JSON – JavaScript Object Notation (текстовий формат обміну даними на основі JavaScript)
6. JWT – JSON Web Token (стандарт для створення токенів доступу)
7. MVC – Model-View-Controller (модель-представлення-контролер, архітектурний шаблон ПЗ)
8. OASIS – Organization for the Advancement of Structured Information Standards (Організація із вдосконалення стандартів структурованої інформації)
9. OAuth – Open Authorization (протокол відкритої авторизації)
10. OIDC – OpenID Connect (протокол автентифікації на основі OAuth 2.0)
11. ORM – Object-Relational Mapper (об’єктно-реляційний мапер)
12. PKCE – Proof Key for Code Exchange (ключ підтвердження для обміну кодом)
13. RFC – Request for Comments (документ із серії технічних стандартів Інтернету)
14. SaaS – Software-as-a-Service (програмне забезпечення як сервіс)
15. SAML – Security Assertion Markup Language (мова розмітки декларації безпеки)

16. SDK – Software Development Kit (набір засобів розробки програмного забезпечення)
17. SIEM – Security Information and Event Management (система управління подіями та інформаційною безпекою)
18. SOLID – абревіатура п'яти основних принципів об'єктно-орієнтованого програмування (принцип єдиної відповідальності, принцип відкритості/закритості, принцип підстановки Лісков, принцип розділення інтерфейсу, принцип інверсії залежностей)
19. SQL – Structured Query Language (мова структурованих запитів)
20. SSO – Single Sign-On (технологія єдиного входу)
21. UI – User Interface (користувацький інтерфейс)
22. UML – Unified Modeling Language (уніфікована мова моделювання)
23. XaaS – Everything-as-a-Service (усе як сервіс)
24. БД – База даних
25. ОЗП – Оперативна пам'ять
26. ПЗ – Програмне забезпечення
27. СУБД – Система управління базами даних
28. SPA – Single Page Application (односторінковий додаток)
29. JWKS – JSON Web Key Set (набір ключів JSON для перевірки підписів JWT)
30. REST – Representational State Transfer (архітектурний стиль взаємодії компонентів розподіленого додатка)
31. GUI – Graphical User Interface (графічний інтерфейс користувача).
32. CSRF – Cross-Site Request Forgery (підробка міжсайтових запитів).
33. DI – Dependency Injection (ін'єкція залежностей).
34. GCP – Google Cloud Platform (хмарна платформа Google).
35. ООП – Об’єктно-орієнтоване програмування.
36. TCP/IP – Transmission Control Protocol / Internet Protocol (набір мережевих протоколів).

<Сторінка змісту>

ВСТУП

Сьогодні більшість програмних продуктів є доступними нам через Інтернет, зокрема з використанням хмарних технологій. Особливо популярність такого підходу зросла із розповсюдженням концепції XaaS (Everything-as-a-Service) – усе як сервіс. Це поняття охоплює загальновідомі моделі, такі як Saas (Software-as-a-Service) – програмне забезпечення як сервіс, IaaS (Infrastructure-as-a-Service) – інфраструктура як сервіс, тощо.

Із поширенням XaaS відповідно значно зросла потреба в передаванні даних користувачів мережею Інтернет та їх обробці і зберіганні на віддалених серверах. Усе це значно ускладнює забезпечення безпеки даних, адже розподілені сервіси, що комунікують через Інтернет, мають набагато більше вразливостей, ніж сервіси, робота яких не виходить за межі одного хосту чи Інтранету. Велике значення у забезпеченні безпеки хмарних сервісів має Identity and Access Management (IAM, керування ідентифікацією та доступом).

Налаштування механізмів авторизації та автентифікації в системах, які складаються з багатьох вузлів, потребує комплексного підходу. У таких середовищах часто виникає необхідність побудови безпечної взаємодії не тільки з кінцевими користувачами, а й з іншими програмними сервісами як всередині системи, так і поза її межами.

Через це для уникнення розрізнених підходів до реалізації IAM в одній системі створено низку стандартів та протоколів. Вони узагальнюють рішення проблеми забезпечення безпеки доступу та надають чіткі норми реалізації складних схем авторизації та автентифікації для різних типів взаємодії в системах. Спираючись на ці правила команди розробників можуть застосовувати випробувані й надійні стратегії захисту своїх програмних продуктів.

Об’єктом цього дослідження є процеси Identity and Access Management у сучасних інформаційних системах. Дослідження спирається на детальний аналіз загальноприйнятих протоколів і стандартів.

Подана кваліфікаційна робота зосереджена передусім на взаємодії кінцевого користувача з додатком напряму та на взаємодії одного додатку з іншим від імені кінцевого користувача. Для демонстрації такої системи буде побудовано декілька вебдодатків на платформі .NET. Предметом дослідження є вивчення та впровадження механізмів автентифікації та авторизації на основі протоколу OpenID Connect (OIDC) – поширеного стандарту сучасних вебсервісів.

Метою поданої роботи є розробка базового програмного рішення для автентифікації та авторизації користувачів у розподілених системах. Рішення має базуватися на доступних ресурсах платформи .NET і відповідати вимогам протоколу OIDC, щоб забезпечити необхідний рівень захисту. Система повинна реалізовувати основні методи управління доступом на основі ролей, а також залишатися розширюваною для можливості подальшої інтеграції більшої кількості схем авторизації та автентифікації.

Для досягнення мети роботи необхідно провести детальний аналіз найсучасніших протоколів та стандартів з автентифікації та авторизації, зокрема OIDC та OAuth 2.0, на якому базується попередній. Разом із тим буде досліджено відкритий стандарт JWT, який буде використано для формування токенів, необхідних для реалізації IAM. Також важливо знайти перевірені методи втілення рішення на платформі .NET. Зокрема слід обрати надійну бібліотеку для роботи з токенами та імплементації необхідних схем авторизації. Крім того, буде проаналізовано можливі сценарії використання рішення для визначення оптимальних методів IAM.

Особливу увагу приділено відповідності програмного рішення вимогам обраних протоколів і стандартів та використанню відкритих, перевірених інструментів для забезпечення надійної та коректної роботи рішення. Отримане рішення є модульним, що забезпечить достатній рівень гнучкості для його подальшого розвитку, розширення та інтеграції у хмарний сервіс.

РОЗДІЛ 1
ДОСЛІДЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПІДХОДІВ ДО РЕАЛІЗАЦІЇ ПРОГРАМНОГО РІШЕННЯ

1.1. Аналіз предметної області

Системи авторизації та автентифікації від часу своєї появи зазнали суттєвих трансформацій. Можна виокремити декілька етапів їх еволюції [1], що відбулися через неминуче ускладнення й масштабування програмної інфраструктури відповідно до зростання потреб користувачів та розвитку архітектури інформаційних систем.

На ранніх етапах існування систем, коли додатки існували ізольовано один від одного, традиційна модель автентифікації за логіном і паролем була достатньою. Але згодом, зокрема у корпоративному середовищі, ефективність такого підходу знизилась: один працівник виконує свою роботу в кількох різних додатках, і в усіх має створювати окремий акаунт. Окрім повторюваності дій з боку користувачів (введення логіну та паролю перед входом до кожного додатка), також зросло адміністративне навантаження, зокрема через потребу в управлінні повним життєвим циклом усіх цих облікових записів. Наявність неактивних, некерованих акаунтів, що можуть з’явитися, наприклад, після звільнення співробітника, значно підвищує ризики інсайдерських та зовнішніх хакерських атак.

Для вирішення проблеми кількох акаунтів, які відповідають одній особі користувача, було введено поняття директорії – сервісу, що централізовано управляв даними усіх користувачів у межах однієї закритої мережі. Таким чином необхідно було лише один раз автентифікуватися у цьому сервісі, і всі внутрішні додатки отримували інформацію про автентифікованого користувача, забезпечуючи Single-Sign-On (SSO). Незадовго після появи директорій з’явилася потреба у новому рівні взаємодії інформаційних систем – міжорганізаційному. Наявне рішення було обмеженим однією внутрішньою мережею, тож директорія була винесена як окремий загальний сервіс, до якого

отримати доступ можна було через Інтернет. Популярні протоколи, які використовувалися для комунікації між директоріями та додатками, висували достатньо високі вимоги до апаратного забезпечення та програмної інфраструктури в цілому, що обмежувало гнучкість цього рішення.

Саме через це почалася робота над новим стандартом IAM, який було названо Security Assertion Markup Language (SAML) і випущено у 2005 р. [2]. Його розробкою займалися Organization for the Advancement of Structured Information Standards (OASIS). Цей протокол визначив базові концепції обміну даними про автентифікацію та авторизацію, на основі яких розвивалися новіші стандарти. В основі SAML лежить передавання спеціальних тверджень (assertions), що містять інформацію про автентифікацію користувача та його права доступу до інформаційних ресурсів, що захищаються.

SAML вирішив багато проблем директорій, проте з'явилася нова вимога, яка допомогла сформувати сучасний вигляд багатьох процесів IAM. Нею стала делегована авторизація, яка відкрила багато функціональних можливостей, але потребувала нового, безпечнішого підходу до реалізації. Делегована авторизація полягає в тому, що користувач дозволяє одній програмі доступ до своїх даних у іншій. На момент появи цієї моделі управління доступом користувач був змушений надавати свої облікові дані сторонній програмі, щоб та могла керувати ресурсами від його імені. Це створювало серйозні ризики. Передусім, пароль користувача надавали сторонньому додатку, і якщо він не міг убезпечити ці дані від атак та витоків, їх могло бути скомпрометовано. Іншою небезпекою було те, що третя сторона мала необмежений у часі та обсязі доступ до ресурсів користувача, що вимагало повної довіри з його боку.

У 2007 було випущено відкритий стандарт OAuth 1.0 (Open Authorization) у вигляді RFC 5829. У 2012 році було випущено його нову версію – OAuth 2.0 (RFC 6749 [3]), яка, хоч і базувалася на досвіді, отриманому після впровадження OAuth 1.0, не була з ним зворотно сумісною і вважається застарілою. Цей стандарт дозволив вирішити проблему передавання облікових даних стороннім програмам, встановлювати обмеження доступу лише до

певних ресурсів і визначати строки дії авторизації через використання токенів. Токен у даному випадку є згенерованим, підписаним цифровим підписом об’єктом, що ідентифікує користувача (чи програму), а також містить дані про час дії дозволів та рівень наданого доступу. Токени, які використовуються в протоколі OAuth 2.0, називаються Bearer Tokens, і їх специфікація подана в окремому RFC 6750 [4], випущеному разом із RFC 6749.

Окрім потреби авторизувати один додаток на доступ до даних іншого, виникла ще одна: надати можливість третій стороні автентифікувати користувача. Це корисно, наприклад, для створення акаунту в новому додатку на основі даних уже наявного облікового запису, або автентифікації в один додаток через інший. Проте OAuth 2.0 не передбачає автентифікацію, і, незважаючи на спроби використати його разом із додатковими процедурами, знадобилося розширення, яким і став OpenID Connect [5]. Цей стандарт було створено у 2014 році організацією OpenID Foundation. Наразі його уже застосовано найбільшими компаніями світу як для захисту власної інфраструктури, так і для надання послуг авторизації та автентифікації своїм користувачам.

1.2 Виявлення проблеми та шляхів її вирішення

OAuth 2.0 та OpenID Connect стали de facto стандартами в розробці розподілених інформаційних систем. Вибір способу імплементації цих стандартів напряму впливає на архітектуру, гнучкість та вартість побудови й підтримки сервісу.

Підходом, що суворо не рекомендується у більшості випадків, є власна імплементація OAuth/OpenID Connect, особливо з додаванням розширень поза межами рекомендованих специфікацій. Якщо компанія-розробник не є провайдером послуг IAM, який би присвячував значні ресурси на підтримку релевантних вимог безпеки та відповідність протоколам і стандартам, процес вибудовування власного рішення швидко стане проблемою. Під час розробки такої системи з нуля висока вірогідність створити *критичні вразливості* або *вийти за рамки підтримуваного функціоналу*, зробивши рішення non-compliant. Отже сама імплементація і підтримка такого рішення швидко стануть *ризикованими, дорогими та трудомісткими.*

З іншого боку, існує безліч професійних рішень від відомих компаній – провайдерів сервісів автентифікації та авторизації, як от Auth0, Okta, Azure Entra Id тощо. Вони беруть значну частину складності реалізації стандартів на себе, однак їх використання також має низку потенційних недоліків, які необхідно зважити та врахувати. Серед них:

а) *Вартість послуг.* Інтеграція із зовнішнім провайдером може стати суттєвим фінансовим навантаженням для бізнесу, якщо він має обмежені ресурси.

б) *Обмежена гнучкість зовнішніх рішень*, що може бути мало помітною на початку, але стати перешкодою для розвитку на пізніших стадіях. Це особливо актуально для великих розподілених сервісів, що з часом розвивають специфічні вимоги до інфраструктури та функціональних можливостей. Усунення таких обмежень часто є складним чи затратним процесом для платформ, що вже активно використовуються.

в) *Зберігання даних, пов'язаних з особою користувача, на серверах третьої сторони.* Хоча для багатьох сервісів це не є проблемою, деякі державні, фінансові чи медичні сервіси можуть підпадати під регуляторні вимоги, що обмежують способи зберігання й обробки таких даних.

г) *Залежність від зовнішніх сервісів.* У разі тимчасової недоступності сервісів провайдера, або якщо він вирішить припинити надання послуг у певному регіоні, наслідки матимуть безпосередній вплив на працездатість і стабільність залежних від нього платформ. Окрім цього, саме провайдер визначає напрямок розвитку функціональних можливостей свого сервісу і умови його використання, що може не враховувати потреби та можливості окремих клієнтів.

Нарешті, є третій підхід, саме якому присвячена подана кваліфікаційна робота – створення гнучкого рішення для IAM на основі бібліотеки. Це дозволить делегувати усю найскладнішу та найризикованішу роботу зі стандартами її творцям. Разом з тим, рішення залишатиметься достатньо модульним та гнучким, щоб задовольняти різні потреби сервісу, в який його вбудовують. За складністю, ризиками та операційною вартістю воно знаходиться між двома іншими, зазначеними вище. Воно все ще потребує ґрунтовних знань про роботу протоколів і стандартів, але вже на набагато вищому рівні.

Подана робота зосереджується на використанні OIDC за схемою Authorization Code Flow, рекомендованою відповідними стандартами для програмних сервісів, що безпосередньо взаємодіють з кінцевими користувачами. Буде розроблено основний додаток (сервер авторизації), який реалізовуватиме OpenID Connect для автентифікації та авторизації користувачів на платформі .NET. Для цього буде використано безкоштовну бібліотеку з відкритим кодом OpenIddict [6], що легко інтегрується у вебдодатки. Окрім цього, буде створено два допоміжні додатки. Перший – сервер ресурсів, що виконуватиме лише верифіковані запити, а другий – клієнт, який запитуватиме дозвіл на доступ до сервера ресурсів. Також буде

розроблено графічний інтерфейс для взаємодії з кінцевими користувачами. Так вони зможуть пройти автентифікацію та підтвердити надання обраних дозволів на доступ до захищених ресурсів.

1.3 Характеристика потенційних користувачів та сфери застосування

Від розробки поданого програмного рішення можуть отримати користь одразу декілька груп користувачів:

а) *Кінцеві користувачі*, що взаємодіють із системою автентифікації та авторизації. Вони зможуть отримати переваги інтуїтивного інтерфейсу та чіткої комунікації з додатком, що особливо важливо, коли мова йдеться про захист їх інформаційних ресурсів.

б) *Розробники ПЗ*, що зможуть використати готове програмне рішення для власного продукту, розширити його та розвивати на основі нього програмну інфраструктуру як у модульних монолітах, так і в мікросервісних архітектурах.

в) *Адміністратори* отримають централізоване сховище даних клієнтських застосунків та користувачів. Вони зможуть налаштовувати параметри безпеки та керувати акаунтами у локальному рішенні, контрольованому їх компанією. Це спростить комунікацію та реалізацію внутрішніх політик управління та безпеки.

г) *Зацікавлені в навчанні розробники*, що ще не мають досвіду роботи з подібними системами авторизації та автентифікації. Вони зможуть отримати доступ до публічного репозиторію, наочно побачити, як такі рішення працюють зсередини, та швидше зрозуміти, як застосовувати їх у власних проєктах.

В залежності від того, для якої інформаційної системи необхідна інтеграція OpenID Connect, можна також виокремити наступні категорії сервісів:

а) *Розподілені та мікросервісні рішення*. Вони отримають користь від існування централізованого вузла автентифікації та авторизації для багатьох

незалежних сервісів. Це допоможе надати плавний досвід SSO користувачам та єдине “джерело правди” щодо облікової інформації.

б) *Корпоративні системи*. Власне рішення для IAM зможе надати таким системам необхідний суверенітет даних, адже не потребує залежності від зовнішніх провайдерів.

в) *SaaS-платформи та стартапи*. Тут перевагою виступає баланс між тривалістю імплементації та фінансовими витратами, що часто є критично важливим, особливо на ранніх стадіях розвитку бізнесу.

г) *Освітні та тестові середовища*, де подане рішення може слугувати шаблоном для навчання принципам роботи OAuth 2.0 з OIDC.

Отже, програмне рішення, розроблене у цій кваліфікаційній роботі, має високу актуальність. Окрім зазначених вище категорій користувачів та систем, що зможуть скористатися цим рішенням, варто виділити декілька додаткових випадків, де рішення також доведе свою користь.

Однією з них є допомога в переході від монолітів до мікросервісів для уже наявних додатків, що потребують модернізації. Це стає можливим завдяки гнучкості програмного рішення.

Завдяки повному контролю над даними, що проходять через сервіс керування доступами, на його основі можна побудувати власну інфраструктуру моніторингу процесів IAM. Це дозволить зібрати достатньо повну інформацію про активність користувачів, а також частоту та успішність запитів на автентифікацію. Окрім того, можна буде виявляти аномальні патерни поведінки, які можуть свідчити про спроби підбору паролів або несанкціонований доступ. Отримані дані можуть бути використані відділом кібербезпеки та інтегруватися із зовнішніми SIEM-системами (Security Information and Event Management) [7] для автоматизованого реагування на загрози.

1.4 Теоретичні засади реалізації механізмів автентифікації та керування доступом

У цьому підрозділі наведено теоретичне обґрунтування вибору моделі виконання автентифікації та авторизації та форматів обміну даними у програмному рішенні.

В основі проєктування розроблюваної системи лежить модель делегованої авторизації, визначена специфікацією RFC 6749 (OAuth 2.0) [3]. Вона виділяє чотири ролі компонентів системи для опису процесів IAM:

а) Власник ресурсів – кінцевий користувач, суб’єкт доступу, який надає доступ до захищених ресурсів.

б) Сервер ресурсів – сервер, на якому знаходяться захищені ресурси.

в) Клієнт – додаток, що намагається отримати доступ до захищених ресурсів від імені власника ресурсів.

г) Сервер авторизації – сервер, що надає токени доступу клієнтам, після успішної автентифікації власника ресурсів та отримання його згоди.

Клієнтів розрізняють передусім в залежності від рівня конфіденційності. Відповідно до цього визначено два типи клієнтів:

а) *Конфіденційний* – додаток, що виконується на захищеному сервері та зданий надійно зберігати секретні дані (наприклад, класичний веб-додаток).

б) *Публічний* – додаток, який працює у відкритому середовищі (браузерні SPA чи мобільні додатки). Такі клієнти не здатні захистити секрети, адже їх код можна переглянути через інструменти розробника чи декомпілювати.

OAuth 2.0 визначає декілька стандартних потоків авторизації (grant types), але у програмному рішення вирішено використовувати саме *потік авторизації з кодом (Authorization Code flow).* Це рекомендований потік авторизації у випадку прямої участі кінцевого користувача, коли є необхідність залучення користувацького агенту, який не може зберігати секрети.

Під час цього потоку клієнт спершу отримує тимчасовий код авторизації з участю користувача, який потім безпечно обмінюється на необхідні токени між серверною (конфіденційною) частиною додатку та сервером авторизації.

Таким чином жоден із токенів не буде передано через користувацький агент (браузер), мінімізуючи ризик перехоплення токенів у його середовищі.
Для подальшого посилення безпеки Authorization Code Flow рекомендовано використовувати розширення PKCE (Proof Key for Code Exchange) [9]. Спочатку його рекомендували передусім публічним клієнтам, але згодом його використання стало хорошою практикою для всіх типів додатків.

PKCE базується на використання двох значень:

а) Code Verifier – унікальний випадковий рядок, який клієнт генерує для кожної спроби авторизації.

б) Code Challenge – версія Code Verifier, перетворена за допомогою хеш-функції, зазвичай SHA-256.

Клієнт спершу надсилає серверу авторизації Code Challenge, і лише під час обміну коду авторизації на токени – Code Verifier. Це унеможливлює атаки з перехопленням коду авторизації у браузері, адже навіть якщо зловмисник перехопить Authorization Code, то не зможе обміняти його на токени, оскільки він не володітиме значенням Code Verifier, відновити яке з перехопленого Code Challenge неможливо.

Однією з великих переваг OIDC/OAuth 2.0 є те, що вони дозволяють чітко окреслити області доступу (Scopes) та твердження (Claims).

Scopes слугують способом обмеження обсягу прав клієнта на доступ до ресурсів. Наприклад, область “resource:read” дозволяє лише читання даних, а область “resource:write” надасть право редагувати дані на сервері ресурсів. Саме за допомогою областей доступу можна реалізувати принцип найменших привілеїв.

Claims описують суб’єкт доступу для шару автентифікації OIDC (наприклад, його ролі та ідентифікаційні дані [5]), метадані токенів (час видачі, час спливання дії тощо) та самі області доступу.

У програмному рішенні форматом основних токенів, що повертаються внаслідок успішного проходження автентифікації та авторизації, є JWT. Цей формат є самодостатнім і переносить у собі усі твердження, які прив’язані до

нього. Структуру JWT можна побачити на рисунку 2.1.

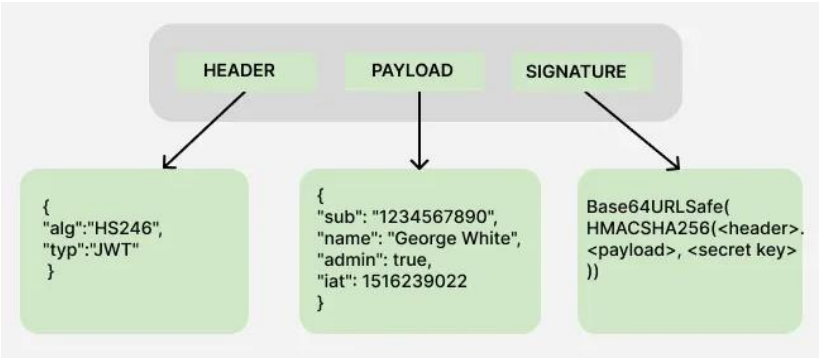


Рис. 2.1 Структура JWT.

Джерело: [10].

У компоненті Header (заголовок) встановлюється тип токену та алгоритм підписання. Payload (тіло) містить набір усіх тверджень відповідно до вимог стандартів, а Signature (підпис) гарантує цілісність даних. Така структура робить JWT stateless: усі дані, необхідні клієнту та серверу ресурсів, вже вкладені у нього, а для його валідації серверу ресурсів необхідно лише перевірити підпис. Для автоматизації процесу валідації токенів використовується стандарт RFC 7517 JWKS (JSON Web Key Set) – публічно доступні відкриті ключі для верифікації, які можна отримати зі спеціальної кінцевої точки сервера авторизації [11].

Для забезпечення повноцінної автентифікації та делегування доступу в системі будуть використовуватися три типи токенів відповідно до OAuth 2.0 та OIDC:

а) Access Token (токен доступу) – надає клієнту право на доступ до захищених ресурсів. Він має обмежений строк дії та зазвичай переноситься в заголовку «Authorization» HTTP-запитів під назвою «Bearer token».

б) Refresh Token (токен оновлення) – випадково згенерований секрет (єдиний з перерахованих не відповідає формату JWT), що використовується винятково для обміну його на новий токен доступу. Його використання не є обов’язковим, але воно значно покращує користувацький досвід (не потрібно часто повторно надавати доступ через сервер авторизації) та дозволяє

скорочувати строк дії токена доступу.

в) ID Token (токен ідентифікації) – специфічний для стандарту OIDC, він містить підтвердження факту автентифікації користувача та його характеристики (у Claims). Він призначений саме для клієнта.

Висновки до Розділу 1

У першому розділі було проведено ґрунтовний аналіз підходів до реалізації Identity and Access Management у сучасних інформаційних системах та їх ключових особливостей. Це дослідження підтверджує роль протоколів OpenID Connect та OAuth 2.0 як фундаменту безпечної автентифікації та авторизації у сучасних програмних сервісах, що підкреслює актуальність поданої кваліфікаційної роботи.

Після огляду можливих підходів до інтеграції згаданих протоколів в інфраструктуру додатків було встановлено, що підтримка балансу між вартістю, складністю та гнучкістю процесів IAM має високу вагу у формуванні правильної архітектури. Рішення, розроблені повністю власноруч, окрім високої вартості підтримки й створення, несуть ризики для безпеки. З іншого боку, використання сервісів сторонніх провайдерів часто обмежує кастомізацію та створює залежність від зовнішніх платформ, а також може негативно позначитися на фінансових витратах. Через це було виявлено потребу у легкому, модульному рішенні, що базується на екосистемі .NET і зможе розвиватися відповідно до потреб інформаційних систем, у які його інтегрують.

Для розв’язання поставленої задачі було сформовано технічне завдання на створення відповідного рішення для автентифікації та авторизації на основі OpenID Connect і OAuth 2.0. Його розробка запланована з січня до квітня 2026 року. У результаті буде отримано основний додаток, що буде реалізовуватиме процеси IAM, і додаток, що буде приймати чи відхиляти запити спираючись

на дані токенів, виданих основним сервером. Також буде створено клієнтський додаток, який допоможе продемонструвати роботу всього рішення.

У розроблюваній програмній компоненті основною метою є впровадження схеми автентифікації та авторизації на основі Authorization Code Flow з PKCE. Для цього було підібрано відповідні засоби створення рішення: платформу .NET, бібліотеку OpenIddict, фреймворки Entity Framework Core та ASP.NET Core Identity, а також реляційну СУБД Microsoft SQL Server. Токени, що використовуватимуться у процесах IAM, будуть формуватися відповідно до стандарту JWT. На додачу буде приділено увагу розробці простого й зрозумілого інтерфейсу, за допомогою якого кінцеві користувачі зможуть зручно управляти своїми даними та дозволами для інших сервісів.

Окреслена архітектура системи передбачає розділення рішення на логічні складові з чіткими зонами відповідальності, такими як підсистеми обробки протокольних запитів, управління обліковими даними користувача та управління сховищем даних. Це допоможе надати рішенню гнучкість, спростити процес його підтримки, а також створити можливість легкого розширення функціоналу.

РОЗДІЛ 2
ПРОЄКТУВАННЯ ВИМОГ ДО РОЗРОБКИ ПРОГРАМНОГО РІШЕННЯ

2.2. Проєктування функціональних вимог

2.2.1. Функціональні характеристики рішення

На основі технічного завдання було визначено перелік ключових функціональних вимог до розробки програмного рішення з автентифікації та авторизації:

Таблиця 2.1.

Функціональні характеристики рішення

Feature / Функціонал	**Опис**
Графічний інтерфейс	Забезпечення інтерфейсом для створення акаунтів, проходження процедури логіну та управління даними.
Підсистема керування користувачами	Підсистема, створена за допомогою ASP.NET Core Identity, що забезпечить роботу з обліковими даними користувачів, ролями, твердженнями (Claims) та їх статусом.
Встановлення та підтримка сесії користувача на сервері авторизації після успішного входу	Сесія користувача на сервері авторизації допоможе знизити частоту, з якою йому необхідно наново вводити свої облікові дані, щоб авторизуватися
Authorization Code Flow	Реалізація відповідно до протоколів OIDC/OAuth 2.0
PKCE	Підсилення потоку додатковим шаром захисту
Стандартні кінцеві точки	Відповідно до обраних протоколів мають бути відкриті необхідні кінцеві точко для взаємодії з іншими додатками

Використання формату JWT для токенів доступу та ідентифікації	Для токенів ідентифікації це дозволить виконати вимогу OIDC, а для токенів доступу це спростить та пришвидшить взаємодію між клієнтом та сервером ресурсів. Такий самодостатній формат токенів дозволяє не робити додаткові запити до сервера авторизації на отримання додаткової інформації про користувача чи обсяги доступу: вона вже вкладена в JWT
Можливість дострокового припинення дії токенів оновлення	Це надає клієнтам можливість відкликати (revoke) токени оновлення, щоб їх вже не можна було використати для обміну на новий токен доступу. Ця практика слідує вимогам, вираженим в RFC 7009 []. Варто зазначити, що відкликання токенів доступу, які мають формат JWT, за замовчанням не проводиться, адже їх стан не прив'язаний нікуди (stateless). Через це, рекомендовано робити строк дії токенів доступу достатньо коротким: 30 хвилин чи менше (згідно з []).
Автоматичне припинення доступу після завершення терміну дії токенів доступу	Завдяки використанню формату JWT, завжди можна буде перевірити, чи дійсний токен, що надіслали на сервер авторизації.
Надання інформації про статус виконаної операції у стандартизованому JSON-форматі з відповідними HTTP кодами	Використання кодів та повідомлень про помилки відповідно до обраних протоколів забезпечить стандартизовані відповіді від сервера авторизації та дозволить реалізовувати передбачувану логіку обробки результатів запитів на стороні клієнтів.
Надання можливості контролю доступу до захищених ресурсів на основі виданих токенів	Для цього необхідно забезпечити сервери ресурсів кінцевою точкою для отримання JWKS, за допомогою даних з якої сервери

	ресурсів зможуть верифікувати отримані в запитах токени доступу. Завдяки використанню JWT у серверів ресурсів не буде необхідності робити додаткові запити до сервера авторизації, щоб дізнатися Scopes та Claims, які відповідають певному токену.
Структуроване збереження даних у реляційній БД	Використання чіткої структури даних та підтримки ACID транзакцій для роботи з даними програмного рішення гарантуватиме цілісність даних.
Хешування паролів	Використання цієї кращої практики дозволяє зберігати паролі у такий спосіб, що навіть за прямого доступу до БД не можна буде відновити їх оригінальні версії

2.2.2. Визначення сценаріїв використання

Розроблюване програмне рішення має декілька основних сценарії використання, описаних у таблиці 2.1.

Таблиця 2.1.

Обґрунтування сценаріїв використання програмного рішення

Сценарій	**Обґрунтування**
Централізоване керування доступом та обліковими даними користувачів	Єдина точка адміністрування облікових даних спрощує масштабування систем та інтеграцію між окремими додатками, а також підвищує загальний рівень безпеки завдяки необхідності контролювати лише один сервер авторизації, що мінімізує поверхню атаки.
Надання делегованого доступу до захищених ресурсів	Протокол OAuth 2.0, на якому побудовано OIDC, дозволяє безпечно надавати делегований доступ без розкриття

	облікових даних користувачів третій стороні.
Автентифікація користувачів	OIDC забезпечує безпечну автентифікацію користувача та передає інформацію про нього в окремому ID-токені, справжність якого можна верифікувати.
Обмеження доступу до ресурсів за ролями та областями доступу	Реалізовує потребу в наданні гранулярного доступу до ресурсів за принципом «найменших привілеїв»: клієнт може отримати доступ лише до тих ресурсів, які необхідні для його роботи.

Джерело: розроблено автором.

2.2.3. Розробка користувацького і програмного інтерфейсів у рішенні

Для повноцінної реалізації рішення було визначено потребу у проєктуванні двох видів інтерфейсу: графічного (GUI) та програмного (API).

Графічний інтерфейс призначений для безпосередньої взаємодії кінцевих користувачів із сервером авторизації. Він має забезпечувати інтуїтивно зрозумілі процеси реєстрації, логіну, керування профілем та надання дозволів під час делегованої авторизації. Вирішено, що інтерфейс буде реалізовано за допомогою технологій ASP.NET Core MVC, зокрема Razor Views. Це спрощує інтеграцію механізмів безпеки, оскільки валідація та захист від типових атак (як-от CSRF) виконуються безпосередньо на сервері під час формування сторінки. Прототип GUI було розроблено у Figma та наведено у додатку Б.

Прикладний програмний інтерфейс (API) виступає основним каналом комунікації між сервісами. API реалізовано відповідно до специфікацій REST та OAuth 2.0/OIDC. Взаємодія здійснюється через набір стандартизованих кінцевих точок, таких як /connect/authorize (для початку авторизації), /connect/token (для отримання токенів) та /.well-known/openid-configuration (для автоматичного виявлення конфігурації сервера). Усі запити та відповіді

мають формат JSON, згідно з принципами REST-архітектури.

2.3. Проєктування нефункціональних вимог

2.3.1. Характеристика основних нефункціональних вимог до рішення

У таблиці 2.2 наведено основні нефункціональні вимоги, висунуті до програмного рішення.

Таблиця 2.2.

Нефункціональні вимоги до програмного рішення

Вимога	**Опис**
Performance	Запити на автентифікацію/авторизацію мають виконуватися не більше 200 мс у середньому (видача токенів має бути достатньо швидкою для безперешкодної інтеграції в роботу з іншими додатками).
Scalability	Система має коректно та ефективно працювати за паралельної взаємодії з мінімум 1000 користувачів з можливістю реалізації подальшого скейлінгу, як вертикального, так і горизонтального.
Security	Паролі мають бути хешовані з salt. Токени мають бути підписані криптографічно. Має підтримуватися PKCE. Час дії токенів має спливати автоматично, refresh tokens мають передаватися безпечно згідно з протоколами. Чутлива конфігурація має зберігатися лише у спеціально відведених окремих файлах. Комунікація має здійснюватися винятково через HTTPS. З’єднання з БД за допомогою зашифрованих каналів.
Fault Tolerance	Програмне рішення має комплексно обробляти виключення, які стаються в коді, щоб не допустити зупинку роботи сервісу.
Data Integrity	Використання транзакцій для взаємодії з БД.
Reliability/ Availability	Рішення має безперебійно працювати 99.9% часу. Мають існувати резервні копії БД для швидкого відновлення від потенційних проблем.
Maintainability/ Extensibility	Модулярний дизайн, використання Dependency Injection, використання кращих практик ООП
Compliance	Рішення повинне відповідати стандартам OAuth 2.0 та OpenID Connect

Portability	Рішення має використовувати лише кросплатформенні бібліотеки для коректної роботи в контейнеризованому середовищі, зокрема на ОС Linux
Deployment/ Operations	Має бути створено публічний репозиторій GitHub з вихідним кодом, документацією та інструкціями, README-файлом та UML діаграмами для візуалізації
Usability/ Ergonomics	Адаптивний інтерфейс, достатньо високий контраст, зрозумілі повідомлення від системи

Джерело: розроблено автором.

2.3.2 Діаграма класів (class diagram)

Для демонстрації внутрішньої побудови ядра програмного рішення – сервера авторизації – було розроблено декілька діаграм класів, кожна з яких відповідає одному з трьох шарів, на які можна логічно розділити цю компоненту: шар контролерів, шар доступу до даних та шар компонентів інтерфейсу підсистеми Identity.

Шар доступу до даних описаний діаграмою на рисунку 2.2. В його основі лежить інтеграція з Entity Framework Core (ORM) та ASP.NET Core Identity.

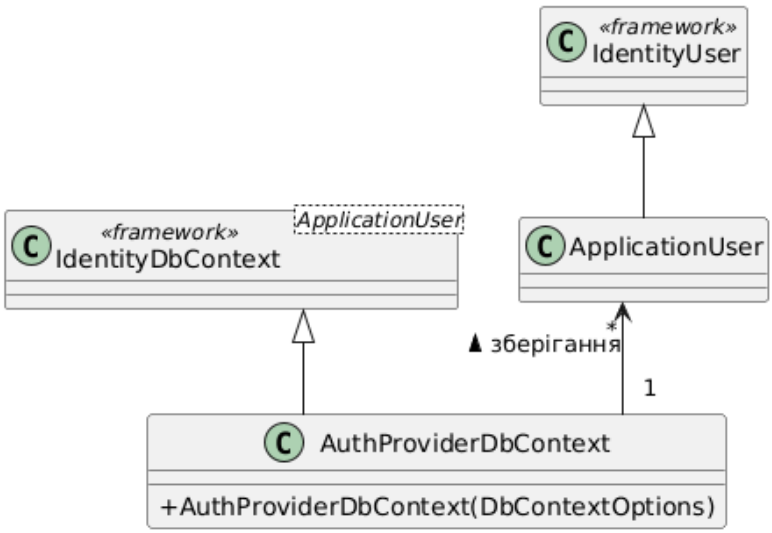


Рис. 2.2. Діаграма класів шару доступу до даних.

Джерело: розроблено автором.

Клас ApplicationUser наслідує від класу IdentityUser, наданого ASP.NET Core Identity. IdentityUser містить основні характеристики користувача, що бібліотека надає за замовчанням. Хоча наразі ApplicationUser не додає нові властивості до цього класу, цей зв'язок створює можливість подальшого розширення сутності без великої кількості модифікацій у коді. AuthProviderDbContext також розширює вбудований клас від Identity Framework, який у свою чергу базується на класах Entity Framework Core і підключає усі конфігурації таблиць для управління ідентичністю користувача за замовчанням.

Діаграма *шару компонентів інтерфейсу підсистеми Identity* (рис. 2.3) описує побудову моделей, що забезпечують роботу сторінок, за допомогою яких користувач може зареєструватися, увійти до свого акаунту та управляти своїми обліковими даними на сервері авторизації.

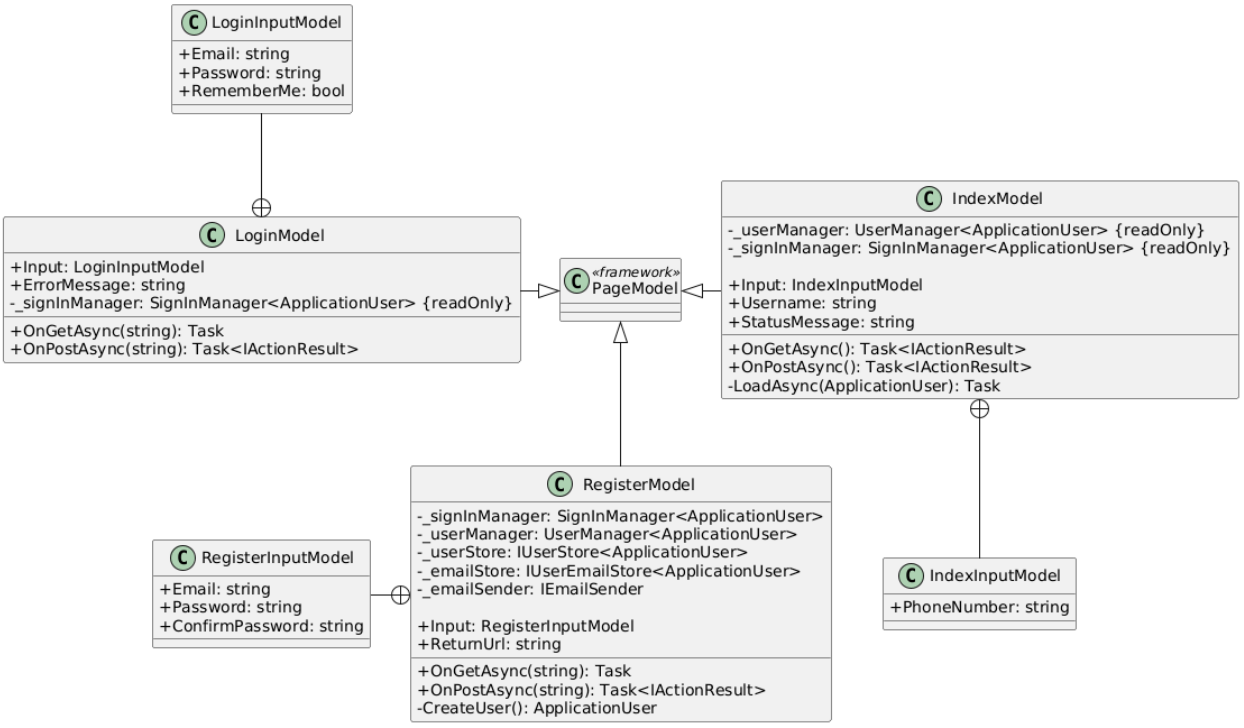


Рис. 2.3. Діаграма класів шару Identity UI.

Джерело: розроблено автором.

Було розроблено 3 моделі: LoginModel, RegistrationModel та IndexMode (призначена для перегляду облікових даних та управління ними). Усі моделі

наслідують від класу PageModel, забезпеченого ASP.NET Core Identity, та оголошують додаткові вкладені моделі (LoginInputModel, RegisterInputModel та IndexInputModel) для взаємодії з формами на відповідних пов'язаних HTML-сторінках.

Для реалізації операцій автентифікації та реєстрації використовуються SignInManager та UserManager, забезпечуваних ASP.NET Core Identity.

Остання діаграма на рисунку 2.4 відображає побудову *шару контролерів*, що реалізує логіку авторизації та автентифікації.

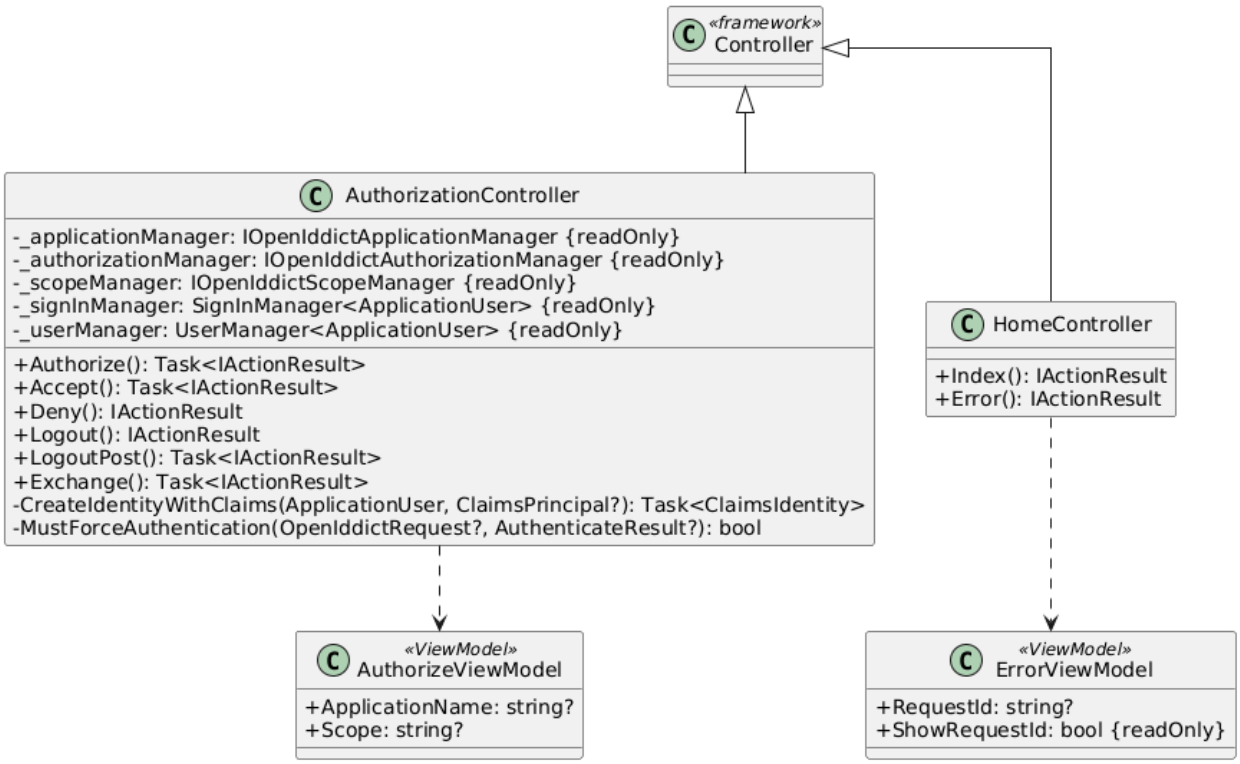


Рис. 2.4. Діаграма класів шару контролерів.

Джерело: розроблено автором.

HomeController є головною сторінкою для входу до інтерфейсу сервера авторизації, а AuthController забезпечує усі процеси OIDC з використанням бібліотек OpenIddict та ASP.NET Core Identity. Кожен контролер взаємодіє з відповідними сторінками HTML і направляє на них визначені View Models.

Можна побачити, що у розроблюваному рішенні більшість механізмів

безпеки та управління ідентичністю користувача надаються обраними бібліотеками. Важливим етапом створення архітектури компоненти є правильна їх конфігурація та інтеграція для реалізації бажаних функцій.

2.3.3. Діаграма послідовності (sequence diagram)

У додатку Б наведено діаграму послідовності, що відображає механізми взаємодії компонентів системи за Authorization Code Flow з PKCE, який лежить в основі програмного рішення. За допомогою цієї схеми можна детально проаналізувати ключові запити та роль кожної компоненти у забезпеченні безпечного процесу автентифікації та делегування доступу.

Взаємодія в системі включає чотири основні актори, які подані в таблиці 2.3. Такий розподіл зумовлено вимогами Authorization Code Flow та архітектурою клієнтського застосунку, що передбачає два типи комунікації: front-channel communication та back-channel communication.

Відповідно до NIST Digital Identity Guidelines [12], front-channel communication – тип взаємодії між двома системами, що спирається на передачу повідомлень через посередника, наприклад через користувацький агент із використанням перенаправлень. Back-channel communication – тип взаємодії між двома системами, що використовує прямий зв’язок без перенаправлень через посередників.

Таблиця 2.3.

Характеристика акторів процесу авторизації/автентифікації

Актор	Опис
Користувач (Resource Owner)	Суб’єкт доступу, що ініціює запит на надання клієнту доступу до своїх захищених ресурсів.
Користувацький агент (User Agent)	Веб-браузер, що виступає посередником для відображення користувацького інтерфейсу та виконання необхідних

	HTTP-перенаправлень відповідно до протоколів.
Клієнт	Конфіденційний клієнтський застосунок, що ініціює запит на отримання доступу й автентифікації, а також відповідає за генерацію значень Code Challenge та Code Verifier для PKCE.
Сервер авторизації	Центральний вузол системи (програмне рішення «AuthProvider»), що відповідає за автентифікацію та авторизацію, валідацію запитів клієнта і випуск токенів.

Процес взаємодії між зазначеними акторами можна розділити на чотири ключові етапи:

а) *Ініціація та підготовка запиту*. На цьому етапі клієнт генерує параметри для PKCE та формує перший запит на сервер авторизації.

б) *Автентифікація користувача.* Сервер авторизації перенаправляє користувача на сторінку автентифікації. Користувач надає свої облікові дані безпосередньо серверу авторизації. Після успішної перевірки даних сервер створює локальну сесію користувача.

в) *Отримання коду авторизації.* Після успішної автентифікації сервер авторизації може додатково запросити у користувача підтвердження обсягу прав доступу (Scopes), якщо вони вимагають його явної згоди. Код авторизації повертається через користувацький агент.

г) *Обмін коду на токени*. Клієнт надсилає запит на сервер авторизації з Authorization Code та Code Verifier. Сервер авторизації валідує Authorization Code, обчислює хеш від наданого Code Verifier і порівнює його з раніше отриманим Code Challenge. За успішного проходження усіх перевірок у відповідь клієнту надаються токени доступу, оновлення та ідентифікації.

2.3.4. Діаграма діяльності (activity diagram)

Для візуалізації алгоритму роботи програмного рішення було розроблено діаграму діяльності (додаток В). Вона відображає логічні операції та демонструє розподіл відповідальності між учасниками процесу.

Процес розділено на три зони відповідальності: користувач, клієнт та сервер авторизації. Виконання основної логіки автентифікації та авторизації зосереджене саме у розроблюваному програмному рішенні (сервері авторизації). В результаті успішного проходження всіх етапів процес завершується наданням токенів клієнту. У разі помилок під час автентифікації користувача чи валідації параметрів запиту на стадії генерації токенів процес призупиняється чи переривається, що гарантує безпеку системи.

2.3.5. Діаграма компонентів (component diagram)

На рисунку 2.5 подано діаграму компонентів програмного рішення.

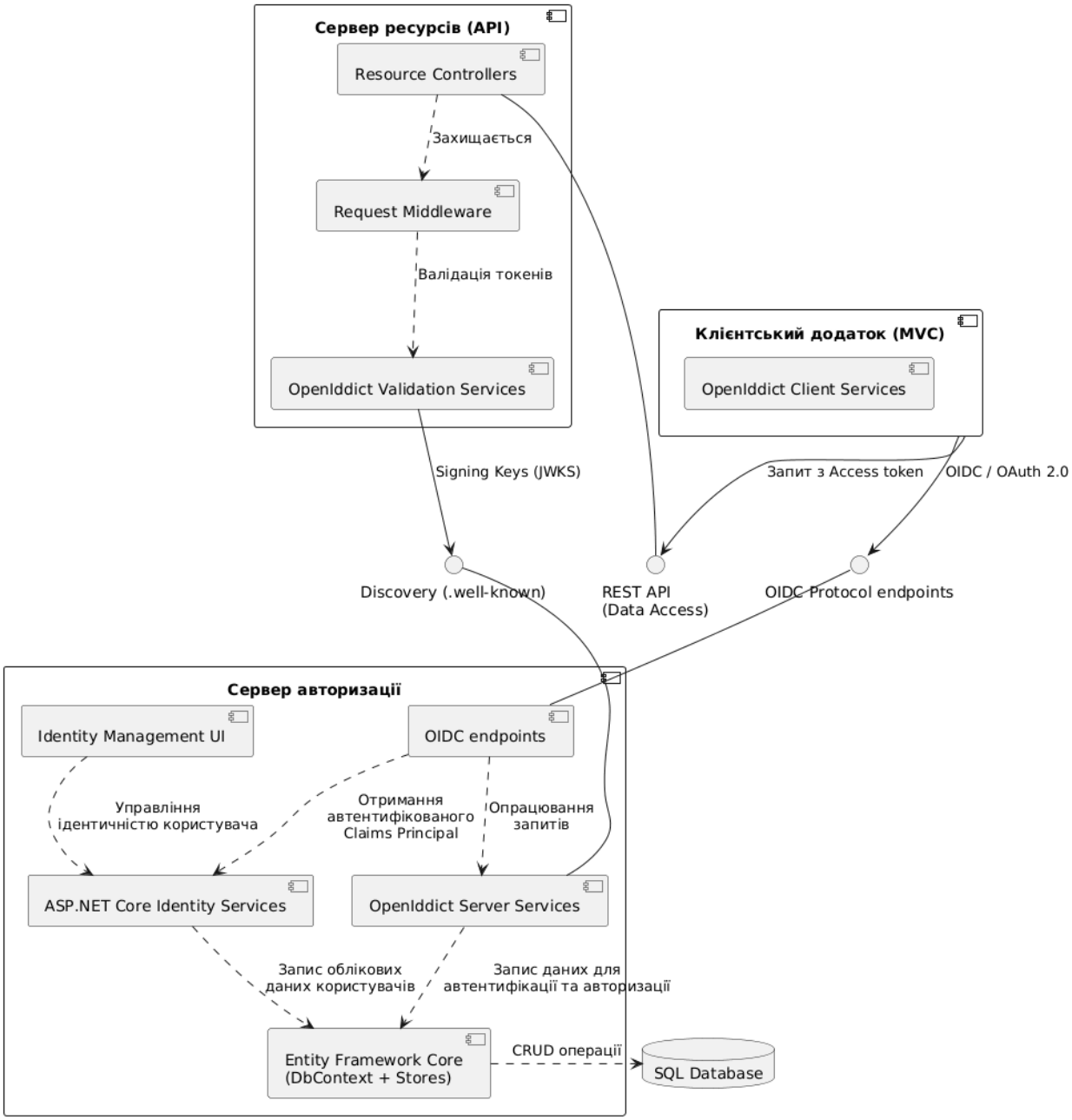


Рис. 2.5. Діаграма компонентів.

Джерело: розроблено автором.

Діаграма компонентів відображає модульну структуру системи і фіксує інтерфейси взаємодії. Вона розглядає як саме програмне рішення, так і допоміжні додатки, показуючи, якими саме способами буде проходити взаємодія між усіма компонентами.

Система складається з трьох основних логічних вузлів та побудована за принципом Separation of Concerns (розділення відповідальності). Кожен компонент розгортається окремо і може масштабуватися незалежно від інших.

Сервер авторизації – єдина точка довіри, і жоден інший компонент не має прямого доступу до облікових даних користувачів та даних про їх автентифікацію та авторизацію.

Клієнт є прикладом класичного клієнт-серверного веб-додатка, який виступає конфіденційним клієнтом. Його взаємодія із сервером авторизації здійснюється винятково через чітко визначені кінцеві точки, що відповідають специфікаціям OIDC/OAuth 2.0. Отримані ним токени використовуються для ідентифікації користувача та доступу до захищених ресурсів.

На сервері авторизації логіка управління користувачами ізольована від логіки реалізації протоколів, але сервіси OpenIddict інтегруються з сервісами ASP.NET Core Identity, зокрема для роботи з Claims Principal для валідації даних та створення токенів. EF Core виступає спільним механізмом доступу до даних. Дані, якими управляють ці бібліотеки, зберігаються на одному екземплярі реляційної БД.

Сервер ресурсів реалізовуватиме захист через валідацію токенів доступу перед обробкою кожного запиту в ASP.NET Core Middleware. Він розглядає сервер авторизації як джерело істини щодо повноважень клієнта.

Взаємодія між усіма компонентами відбувається через визначені інтерфейси. Сервер авторизації забезпечує протокольні кінцеві точки для реалізації IAM та автоматичного отримання конфігурації (Discovery endpoints), а клієнт взаємодіє з сервером ресурсів через REST API.

2.3.6. Діаграма розгортання (deployment diagram)

Нижче (рис. 2.6) наведено діаграму розгортання програмного рішення (Auth Service) та додаткових компонент (Resource Server та Client Service).

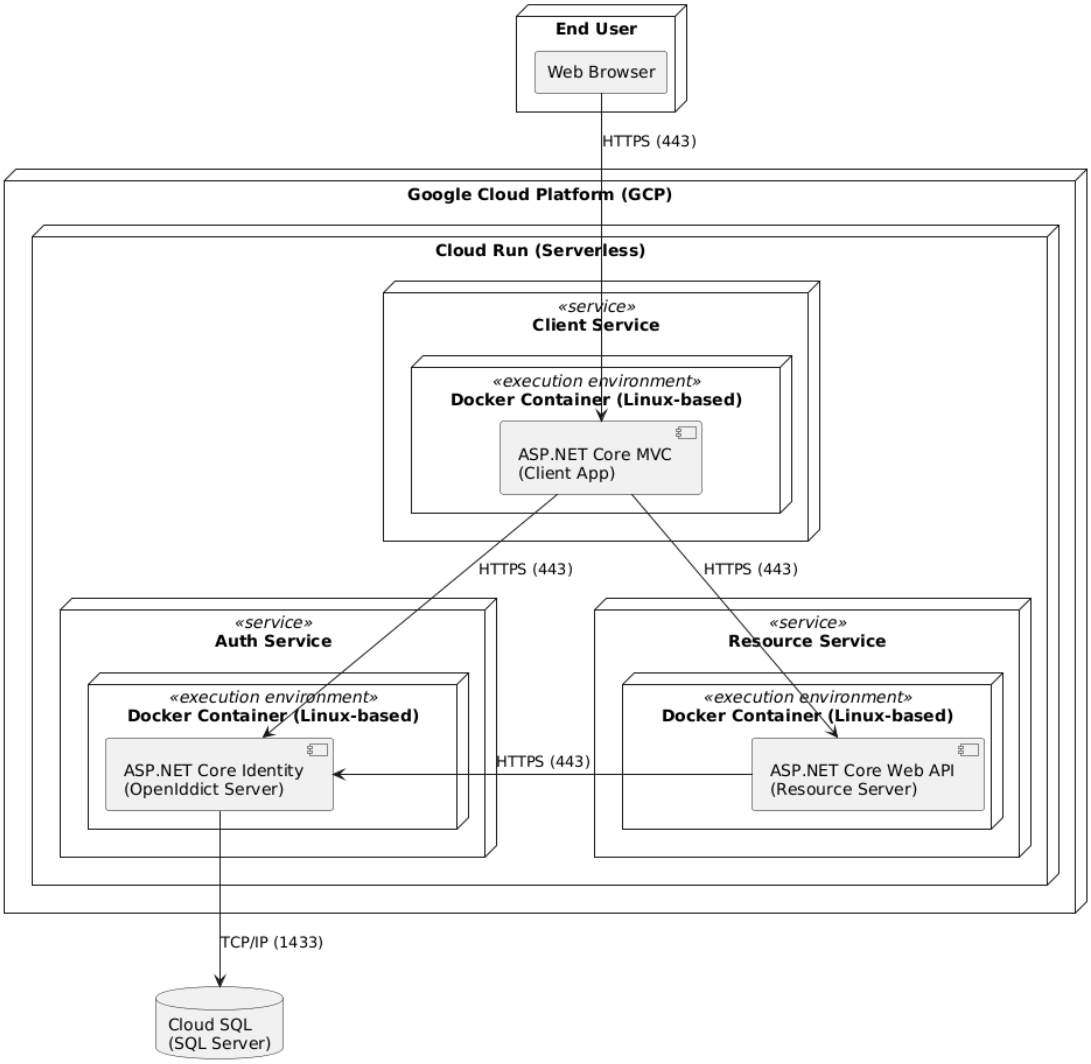


Рис. 2.6. Діаграма розгортання.

Джерело: розроблено автором.

На діаграмі відображено фізичне розміщення компонентів системи та мережеві зв’язки між ними. Обрана архітектура базується на хмарній платформі Google Cloud Platform (GCP). Основою інфраструктури є середовище Cloud Run, яке забезпечує роботу сервісів у безсерверному (serverless) режимі. Кожен вузол розгорнутий як незалежний Docker-контейнер на базі ОС Linux. Таким чином гарантується ізоляція та стабільність середовища виконання коду. Для збереження даних використовується екземпляр SQL Server у межах сервісу Cloud SQL. Комунікація між вузлами здійснюється за допомогою захищених протоколів HTTPS та TCP/IP.

GCP забезпечує ефективне горизонтальне масштабування рішення до

нуля (ресурси не споживаються за відсутності запитів), що дозволяє оптимізувати витрати і швидко реагувати на підвищення навантаження, динамічно створюючи нові екземпляри сервісів відповідно до налаштувань.

Висновки до Розділу 2

У цьому розділі було встановлено основні вимоги до програмного рішення з автентифікації та авторизації “AuthProvider”.

Відповідно до функціональних вимог, рішення реалізовує Authorization Code Flow – найбільш захищений потік протоколів OIDC/OAuth 2.0 для сценаріїв, що передбачають безпосередню участь користувача. Додатковий захист забезпечує використання PKCE. Програма реалізує роботу з протоколами через використання відкритої бібліотеки OpenIddict, а також надає інфраструктуру для роботи з даними користувачів через фреймворк ASP.NET Core Identity. Рішення реалізовано у вигляді додатка на базі ASP.NET Core MVC з програмним та графічним інтерфейсами.

До розробки також було висунуто низку нефункціональних вимог, що охоплюють критерії безпеки, надійності, масштабованості, портативності, продуктивності тощо.

На основі встановлених вимог було розроблено архітектуру програмного рішення, візуалізовану у вигляді ряду UML-діаграм. Вони описують механізми інтеграції компонентів, роботу з обраними бібліотеками, інфраструктуру та ключові алгоритми роботи відповідно до стандартів OIDC/OAuth 2.0. Додатково описано спосіб розгортання рішення на хмарній платформі GCP з використанням технології контейнеризації Docker.

РОЗДІЛ 3
РОЗРОБКА ПРОГРАМНОГО ПРОДУКТУ

3.1. Середовище розробки та стек використаних технологій.

Основна частина роботи над програмним рішенням та демонстраційними додатками відбувалася у середовищі Microsoft Visual Studio 2026. Розробка виконувалася з використанням .NET 10 мовою C#. Такий технологічний стек надає єдину екосистему для реалізації як клієнт-серверних MVC-додатків із SSR (Server Side Rendering), так і Web API.

У процесі розробки як СУБД використовувалася Microsoft SQL Server 2019 (Express Edition) у конфігурації LocalDB, що надає низку переваг перед використанням повноцінного екзмепляра SQL Server Express на етапі реалізації БД.

LocalDB створено саме з метою забезпечити розробників простішою версією СУБД, яку можна легко встановити і швидко почати використовувати. Через це вона набагато менше використовується у production середовищах і має певні обмеження []. Екземпляр LocalDb містить мінімальний набір файлів, необхідних для запуску SQL Server Database Engine. Завдяки своїй полегшеній архітектурі, екземпляр LocalDB функціонує як процес, а не як системна служба (service), на відміну від повноцінних екзмеплярів SQL Server. Цей процес запускається лише за умови наявності активних з’єднань із базою даних, що дозволяє суттєво економити ресурси комп’ютера на етапі розробки.

Додатково для контролю стану бази даних та інспектування створених таблиць використовувалося середовище SQL Server Management Studio 20 (SSMS). Документація до програмного рішення у вигляді UML діаграм була виконана у середовищі Visual Studio Code з технологією PlantUML.

Для контейнеризації компонентів розробленої системи було використано Docker та додатково Docker Desktop для коректної та зручної роботи на ОС Windows 11.

Упродовж розробки програмного рішення для контролю версій та документації застосовувався Git, а результати роботи зберігалися у віддаленому репозиторії, доступ до якого можна отримати за посиланням: https://github.com/AyTee001/AuthProvider.

3.2. Характеристика результатів розробки рішення

У результаті виконання роботи реалізовано систему автентифікації та авторизації, що має багатопроєктну архітектуру. Рішення складається з чотирьох компонентів: AuthProvider – сервер для автентифікації та авторизації; AuthProvider.Client – клієнтський застосунок; AuthProvider.ResourceServer – сервер ресурсів; AuthProvider.Migrator – спеціальний допоміжний додаток для роботи з міграціями, розроблений для БД проєкту AuthProvider.

Такий підхід дозволяє продемонструвати повний цикл взаємодії між учасниками протоколів OAuth 2.0 та OIDC у межах потоку Authorization Code з PKCE, включаючи автентифікацію користувача, видачу токенів та їх подальшу валідацію під час доступу до захищених ресурсів.

Центральною точкою взаємодії зі сховищем даних у проєктах AuthProvider та AuthProvider.Client є контекст бази даних Entity Framework Core. Для керування розвитком схем БД у рішенні застосовано механізм міграцій за підходом Code-First, що дозволяє інтегрувати сутності з бібліотек у схему БД та керувати їхньою конфігурацією безпосередньо у коді.

Проєкт AuthProvider.Client використовує БД у пам’яті (in-memory database), що обумовлено відсутністю потреби тривалого зберігання даних у цьому демо-застосунку. Дані автентифікації та авторизації у ньому зберігаються в Cookies.

Важливу роль у налаштуванні БД сервера авторизації відіграє проєкт AuthProvider.Migrator. Це консольний додаток, метою якого є надання способу легко створювати та застосовувати міграції Entity Framework Core. Генерувати міграції можна з командного рядка. Це можливо завдяки спеціальному класу

(AuthProviderMigratorDbContextFactory), який реалізує інтерфейс IDesignTimeDbContextFactory.

Реалізація AuthProvider.Migrator дозволила відділити логіку управління схемою БД від логіки процесів IAM. Це допомагає реалізувати розділення відповідальностей та спрощує процес розгортання рішення, а також його горизонтального масштабування. Так якби логіка роботи зі станом БД була вбудована безпосередньо в основний додаток (сервер OIDC), кожен новий його екземпляр намагався б застосувати найсвіжіші зміни до БД, що могло б провокувати конфлікти та помилки.

Під час запуску AuthProvider.Migrator створює БД, якщо її ще не існує, та застосовує до неї актуальні міграції. Крім того, AuthProvider.Migrator у разі відсутності записує у таблиці OpenIddict дані демонстраційного клієнта. Це забезпечує реєстрацію клієнта, що є вимогою протоколу OAuth 2.0. Таким чином сервер OIDC зможе ідентифікувати клієнтські застосунки та застосовувати встановлені для них правила і обмеження.

Варто зазначити, що такий механізм реєстрації обрано саме для спрощення імплементації на поданому етапі розробки. При подальшій інтеграції програмного рішення у реальне середовище, реєстрацію клієнтів доцільно реалізувати через спеціальні адміністративні панелі управління (Dashboard) або за допомогою протоколу динамічної реєстрації клієнтів (Dynamic Client Registration), в залежності від специфіки та вимог безпеки конкретної системи.

Для взаємодії з сервером ресурсів у проєкті AuthProvider.Client створено типізований HttpClient. Він використовує спеціальний AuthDelegatingHandler, який автоматично додає необхідний заголовок з токеном авторизації для запитів до сервера ресурсів. Крім того, він реалізує механізм обміну токена оновлення на новий набір токенів, якщо термін дії токена доступу минув, повторюючи після цього запит до сервера ресурсів. Таким чином забезпечується безперебійність комунікації клієнта із сервером ресурсів без додаткового втручання користувача.

На сервері OIDC та клієнті активно використовуються механізми ASP.NET Core Data Protection. Зокрема, виконується перевірка Antiforgery Tokens для попередження атак типу CSRF (Cross-site request forgery), а Cookies шифруються та підписуються. Також у додатках використовуються механізми локальних перенаправлень за замовчанням, щоб запобігти атакам типу Open Redirect, а коли є необхідність перенаправлення назовні, ця задача передається внутрішнім механізмам OpenIddict, що для цього використовують дані реєстрації клієнта.

Сервер ресурсів реалізовано як простий ASP.NET Core Web API з одним контролером (JokesController), який надає клієнту випадковий жарт про програмування. Використання атрибуту Authorize на рівні контролера надає захист від неавторизованих клієнтів.

Відповідно до ролей проєктів AuthProvider, AuthProvider.Client та AuthProvider.ResourceServer у процесах автентифікації та авторизації було виконано конфігурацію OpenIddict у кожному з них.

У розробленій MVP-версії програмного рішення також включено ресурси для локального розгортання системи, яка може слугувати основою для подальшої реалізації розгортання у хмарному середовищі. Інструкції для контейнеризації кожного проєкту виконані у відповідних файлах Dockerfile.

Автоматизація збірки контейнерів здійснюється за допомогою інструменту Docker Compose (файл docker-compose.yaml). Для безперешкодної роботи в локальному середовищі контейнеризовані додатки комунікують між собою вільно через localhost, доступ до якого надано за допомогою налаштування “extra_hosts”. Сервер БД працює в окремому контейнері, але записи зберігаються завдяки використанню тому (volume), що управляється Docker.

На даному етапі ізоляція додатків у контейнерах дозволяє протестувати їх роботу в ізольованому детерміністичному середовищі, а також закласти ґрунт для майбутньої хмарної інтеграції через підтримку технологічної відповідності потребам такого середовища.

3.3. Демонстрація та аналіз роботи програмного рішення.

Для наочної демонстрації роботи програмного рішення варто розглянути логи, які забезпечуються OpenIddict та виводяться у консоль під час роботи додатків. Нижче наведено аналіз основних частин процесу автентифікації та авторизації демонстраційного клієнтського додатка на сервері OIDC, де увагу зосереджено на ключових етапах протокольної взаємодії.

Коли користувач розпочинає процес логіну, натиснувши відповідну кнопку в графічному інтерфейсі, клієнт надсилає перший запит на авторизацію, вміст якого подано на рисунку 3.1.

У запит включено ідентифікатор клієнта, посилання для перенаправлення з кодом авторизації (заздалегідь зареєстроване як валідне для поданого клієнта на сервері OIDC), а також для параметра “response_type” встановлено значення “code”, що відповідає обраному Authorization Code Flow. Серед запитаних областей доступу присутня область openid, без якої шару OIDC не було б поверх звичайного потоку авторизації, а також область offline_access, яка просить надати можливість працювати з токенами оновлення.

```
info: OpenIddict.Server.OpenIddictServerDispatcher[6030]
      The authorization request was successfully extracted: {
        "client_id": "mvc-client",
        "redirect_uri": "https://localhost:7197/callback/login/local",
        "response_type": "code",
        "scope": "openid email profile offline_access",
        "nonce": "1YD1eMTWLodRmtWEMZ0zu2dP0AbBpGMBCM_uh0JO48A",
        "code_challenge": "lsHHqnjcN3w-_K0w4Hf_voRpFQNsHolT3mnAL_dfjZo",
        "code_challenge_method": "S256",
        "state": "0lDWzEvK8wgsdFBcnluSoMd5MimjEPJfiwDdPuuFksY"
      }.
```

Рис. 3.1. Діаграма класів шару контролерів.

Джерело: розроблено автором.

Окрім цього запит включає декілька надважливих параметрів для

забезпечення безпеки процесів OIDC/OAuth 2.0. Першим із них є параметр nonce (“number used once”), використання якого специфічне саме для OIDC. Він допомагає захистити клієнтські додатки від атак типу Token Replay: клієнтський додаток не прийме токен ідентифікації, якщо значення параметру nonce, вкладене в нього як claim, не співпаде з тим, яке клієнт встановив на початку усього процесу.

Наступним захисним параметром є state. Він використовується для попередження атак типу CSRF під час процесу авторизації, щоб злочинці не могли підробити запит з перенаправленням, що містить код авторизації. Нарешті, code_challenge та code_challenge_method є частиною захисного шару PKCE.

Після ініціації процесу входу а акаунт, автентифікації користувача на стороні сервера OIDC, та отримання його згоди на надання доступу до захищених ресурсів, клієнту перенаправляється запит з код авторизації (рис. 3.2), а також параметрами state та iss (Issuer – адреса серверу, що видає токени) для верифікації запиту. Параметр iss, описаний в RFC 9207 [], додає шар захисту від атак типу Mix-Up, під час яких клієнта можуть спровокувати надіслати код іншому серверу авторизації, який захопив зловмисник. Цей вид атак стає можливим, коли клієнт починає працювати з більше ніж з одним сервером авторизації.

```
info: OpenIddict.Client.OpenIddictClientDispatcher[6178]
      The redirection request was successfully extracted: {
        "code": "[redacted]",
        "state": "0lDWzEvK8wgsdFBcnluSoMd5MimjEPJfiwDdPuuFksY",
        "iss": "https://localhost:7082/"
      }.
```

Рис. 3.2. Діаграма класів шару контролерів.

Джерело: розроблено автором.

Отримавши код авторизації, клієнт нарешті може обміняти його на токени, і для цього надсилає запит, наведений на рисунку 3.3, що включає сам код, ідентифікатор клієнта, назву типу гранту і посилання для для

перенаправлення (воно повинно співпасти з тим, яке було зазначене у найпершому запиті на сервер OIDC). Окрім цього запит містить секрет клієнта, адже, згідно з вимогою RFC 6749, конфіденційні клієнти повинні автентифікуватися під час запитів на кінцеві точки для роботи з токенами []. Також на цьому етапі в запит включено code_verifier, щоб здійснити перевірку отриманого раніше code_challenge.

```
info: OpenIddict.Server.OpenIddictServerDispatcher[6075]
      The token request was successfully extracted: {
        "grant_type": "authorization_code",
        "code": "[redacted]",
        "code_verifier": "aAceS3Q3s8mZtj0_ZCIzgKuUVatYbleUqEI4B-EiQEs",
        "redirect_uri": "https://localhost:7197/callback/login/local",
        "client_id": "mvc-client",
        "client_secret": "[redacted]"
      }.
```

Рис. 3.3. Діаграма класів шару контролерів.

Джерело: розроблено автором.

У відповідь на попередній запит, клієнт отримує відповідь з усіма токенам, а також додаткові дані: тип токена доступу (Bearer), його строк дії та список областей доступу, що клієнт дійсно отримав після успішної авторизації.

```
info: OpenIddict.Client.OpenIddictClientDispatcher[6193]
      The token response returned by https://localhost:7082/connect/token was
 successfully extracted: {
        "access_token": "[redacted]",
        "token_type": "Bearer",
        "expires_in": 599,
        "scope": "openid email profile offline_access",
        "id_token": "[redacted]",
        "refresh_token": "[redacted]"
      }.
```

Рис. 3.4. Діаграма класів шару контролерів.

Джерело: розроблено автором.

Після завершення процесів автентифікації та авторизації клієнт окремо робить запит до кінцевої точки userinfo сервера OIDC, щоб отримати усі доступні твердження (Claims) про користувача з використанням токену

доступу (рис. 3.5).

```
info: OpenIddict.Client.OpenIddictClientDispatcher[6194]
      The userinfo request was successfully sent to https://localhost:7082/co
nnect/userinfo: {}.
info: OpenIddict.Client.OpenIddictClientDispatcher[6195]
      The userinfo response returned by https://localhost:7082/connect/userin
fo was successfully extracted: {
        "sub": "0505fd2a-9045-4622-8520-5ff9527cfa93",
        "name": "name@gmail.com",
        "email": "name@gmail.com",
        "email_verified": false
      }.
```

Рис. 3.5. Діаграма класів шару контролерів.

Джерело: розроблено автором.

Коли строк дії токена доступу добігає кінця, клієнт за допомогою імплементованого DelegatingHandler може отримати новий, надіславши запит з токеном оновлення, параметром grant_type, встановленим на “refresh_token”, і своїми обліковими даними для автентифікації (рис. 3.6).

```
info: OpenIddict.Client.OpenIddictClientDispatcher[6192]
      The token request was successfully sent to https://localhost:7082/connect/token: {
        "grant_type": "refresh_token",
        "refresh_token": "[redacted]",
        "client_id": "mvc-client",
        "client_secret": "[redacted]"
      }.
```

Рис. 3.6. Діаграма класів шару контролерів.

Джерело: розроблено автором.

Токен ідентифікації у рішення імплементовано у форматі JWS. Разом із тим для представлення токену доступу OpenIddict використовує JWE (JSON Web Encryption, []) із вкладеним усередину JWS. Таким чином забезпечується не тільки цілісність цього токена, але і його конфіденційності, адже зашифрований токен, на відміну від звичного JWS, не можна переглянути, не маючи ключа для розшифрування.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. OAuth2.0 and OpenID Connect: The Professional Guide [Електронний ресурс] // Auth0. – Режим доступу: https://auth0.com/resources/ebooks/oauth-openid-connect-professional-guide

2. Security Assertion Markup Language (SAML) v2.0. [Електронний ресурс] // OASIS Open. – Режим доступу: https://www.oasis-open.org/standard/saml/

3. Request for Comments: 6749. The OAuth 2.0 Authorization Framework. [Електронний ресурс] // Internet Engineering Task Force (IETF). – Режим доступу: https://datatracker.ietf.org/doc/html/rfc6749

4. Request for Comments: 6750. The OAuth 2.0 Authorization Framework: Bearer Token Usage. [Електронний ресурс] // Internet Engineering Task Force (IETF). – Режим доступу: https://datatracker.ietf.org/doc/html/rfc6750

5. OpenID Connect Core 1.0 incorporating errata set 2. [Електронний ресурс] // OpenID Foundation. – Режим доступу: https://openid.net/specs/openid-connect-core-1_0.html

6. OpenIddict Documentation. [Електронний ресурс] // OpenIddict. – Режим доступу: https://documentation.openiddict.com/

7. Security information and event management. [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Security_information_and_event_management

8. Request for Comments: 7519. JSON Web Token (JWT). [Електронний ресурс] // Internet Engineering Task Force (IETF). – Режим доступу: https://datatracker.ietf.org/doc/html/rfc7519

9. Request for Comments: 7636. Proof Key for Code Exchange by OAuth Public Clients. [Електронний ресурс] // Internet Engineering Task Force (IETF). – Режим доступу: https://datatracker.ietf.org/doc/html/rfc7636

10. JSON Web Token (JWT). [Електронний ресурс] // GeeksforGeeks. – Режим доступу: https://www.geeksforgeeks.org/web-tech/json-web-token-jwt/

11. Request for Comments: 7517. JSON Web Key (JWK). [Електронний ресурс] // Internet Engineering Task Force (IETF). – Режим доступу: https://datatracker.ietf.org/doc/html/rfc7517

12. NIST SP 800-63-4. Digital Identity Guidelines. [Електронний ресурс] // National Institute of Standards and Technology. – Режим доступу: https://doi.org/10.6028/NIST.SP.800-63-4

13. Request for Comments: 7009. OAuth 2.0 Token Revocation. [Електронний ресурс] // Internet Engineering Task Force (IETF). – Режим доступу: https://www.rfc-editor.org/rfc/rfc7009

14. Antipattern: Set a long expiration time for OAuth tokens. [Електронний ресурс] // Google Cloud. – Режим доступу: https://docs.cloud.google.com/apigee/docs/api-platform/antipatterns/oauth-long-expiration

ДОДАТКИ

Додаток А

1. Загальні відомості

1.1. Найменування рішення

Повне найменування: «Програмне рішення для автентифікації та авторизації на платформі .NET».

Скорочене найменування: «AuthProvider».

1.2. Планові терміни початку та закінчення робіт

Початок: січень 2026 року.

Завершення: квітень 2026 року.

1.3. Порядок оформлення і пред’явлення результатів робіт

Результат має бути представлено у вигляді вихідного коду вебдодатку на базі ASP.NET Core MVC з API, що реалізує автентифікацію користувачів за протоколом OpenID Connect та делеговану авторизацію на основі OAuth 2.0, разом з UML-діаграмами як документацією схем взаємодії з програмним рішенням. Також буде представлено два додатки, що допоможуть наочно продемонструвати роботу основного API автентифікації та авторизації.

1.4. Головний бенефіціар та потенційні користувачі

Головний бенефіціар – розробники програмного забезпечення, що створюють або інтегрують власні рішення для IAM.

Потенційні користувачі – розробники ПЗ, адміністратори систем, користувачі хмарних платформ.

2. Мета та призначення створення рішення

2.1. Призначення рішення

Проведення автентифікації та авторизації за протоколами OpenID Connect та OAuth 2.0 з використанням найпопулярніших схем, описаних у відповідних специфікаціях протоколів.

2.2. Мета створення рішення

Забезпечення основи для використання в інформаційних системах, а також подальшого розширення функціоналу програмної компоненти, наприклад, додавання нових схем, розширення кількості ролей тощо.

Для імплементації рішення будуть використані наявні бібліотеки з високими можливостями кастомізації та надійністю. Рішення буде окремим додатком, комунікація з яким здійснюватиметься за допомогою мережевого протоколу HTTP(S).

3. Вимоги до рішення

3.1. Вимоги до рішення в цілому

Вимоги до структури та функціонування рішення:

а) Реалізація схем автентифікації та авторизації відповідає вимогам OpenID Connect та OAuth 2.0.

б) Рішення забезпечує базовий інтерфейс для взаємодії з кінцевими користувачами.

в) Використовуються стандартизовані кінцеві точки для прийому та обробки запитів.

д) Використання JWT [8] як формату токенів доступу та ідентифікації.

е) Сервер ресурсів проводить валідацію токенів перед наданням доступу на запит клієнтів.

Показники призначення:

а) Забезпечення повного циклу автентифікації та авторизації зі схемою Authorization Code Flow.

б) Автоматичне припинення доступу після спливання терміну дії токена, можливість перервати дію токена перед закінченням початкового строку його дії.

Вимоги до надійності:

а) Перевірка цілісності токенів за допомогою цифрового підпису.

б) Стійкість до некоректних запитів та інформування за допомогою стандартизованих кодів та повідомлень про природу помилки.

в) База даних для збереження даних, що мають встояти під час тимчасових пауз у роботі рішення.

д) Використання PKCE [9] для схеми з кодом авторизації.

Вимоги до ергономіки та технічної естетики:

а) Адаптивність користувацького інтерфейсу під різні розміри екранів.

б) Достатній контраст для зручного читання інформації, мінімалістичний інтерфейс.

в) Прості переходи між екранами, чіткий зворотний зв’язок під час процесу автентифікації та авторизації у вигляді повідомлень про успіх чи помилки, а також детальний опис помилок.

г) Розбиття коду рішення на чіткі модулі, наслідування принципів SOLID під час розробки для забезпечення легкої підтримки коду.

д) Єдність стилістичного оформлення інтерфейсу.

Вимоги до експлуатації, технічного обслуговування, ремонту і зберігання компонентів системи: рішення можна завантажити з відкритого репозиторію GitHub, разом із документацією з інструкцією щодо встановлення та використання, діаграмами роботи реалізованих алгоритмів авторизації та автентифікації.

Вимоги до захисту інформації:

а) Використання шифрування під час збереження чутливих даних у БД.

б) Хешування секретів клієнтів, а не їх пряме зберігання.

в) Зберігання чутливої конфігурації у відповідних окремих файлах, а не всередині коду.

г) Використання лише HTTPS для комунікації між компонентами рішення.

д) Мінімізація кількості чутливих даних, що зберігаються.

3.2. Перелік підсистем компоненти

а) Модуль обробки протокольних запитів згідно з OIDC.

б) Підсистема управління особами користувачів: автентифікація з графічним інтерфейсом, зберігання даних користувачів та їхніх повноважень, взаємодія з БД для перевірки облікових даних.

в) Підсистема збереження даних: збереження метаданих клієнтських застосунків, токенів та інформації про користувачів, автоматичне очищення видалених токенів, абстракція доступу через ORM.

г) Підсистема захисту та надання доступу (для серверу ресурсів): валідація токенів, контроль доступу в залежності від дозволів.

3.3. Вимоги до видів забезпечення

Вимоги до математичного забезпечення:

а) використання алгоритмів криптографічного підпису для токенів;

б) використання алгоритмів шифрування та хешування для даних у БД;

в) врахування часових міток для параметрів часу видачі та часу спливання дії у токенах;

г) для генерації усіх токенів використовуються відповідні стандартизовані методи.

Вимоги до інформаційного забезпечення:

а) реалізація усіх компонентів рішення має спиратися на відповідні стандарти та протоколи;

б) форматом передачі даних має бути JSON;

в) у коді та на рівні API дані мають подаватися у вигляді суворо типізованих моделей;

д) використання форматів даних, які доцільно зберігати у БД (наприклад, VARCHAR(n) замість VARCHAR(MAX) там, де максимальна довжина даних n може бути чітко визначена);

3.4. Вимоги до програмного забезпечення

а) Середовище виконання: .NET 8.

б) Розробка мовою C#.

в) Об’єктно-реляційний мапер: Entity Framework Core.

г) СУБД: Microsoft SQL Server.

д) Бібліотека для реалізації управління користувачами: ASP.NET Core Identity.

е) Основна бібліотека для реалізації процесів IAM: OpenIddict.

є) Вебдодатки має бути реалізовано на ASP.NET Core MVC.

3.5. Вимоги до технічного забезпечення

а) Мінімальні вимоги до середовища: має бути встановлено .NET SDK 8, MS SQL Server Express 2025, веб браузер.

б) Для нормальної роботи самого рішення без урахування потреб додаткових програм для його функціонування необхідно щонайменше 1 ГБ ОЗП.

3.6. Вимоги до методичного забезпечення

Має бути створено:

а) технічна документація щодо встановлення та використання рішення;

б) діаграми з ілюстрацією принципів роботи рішення.

в) README.md з коротким представленням рішення та його особливостей і короткою навігацією по репозиторію.

Додаток Б

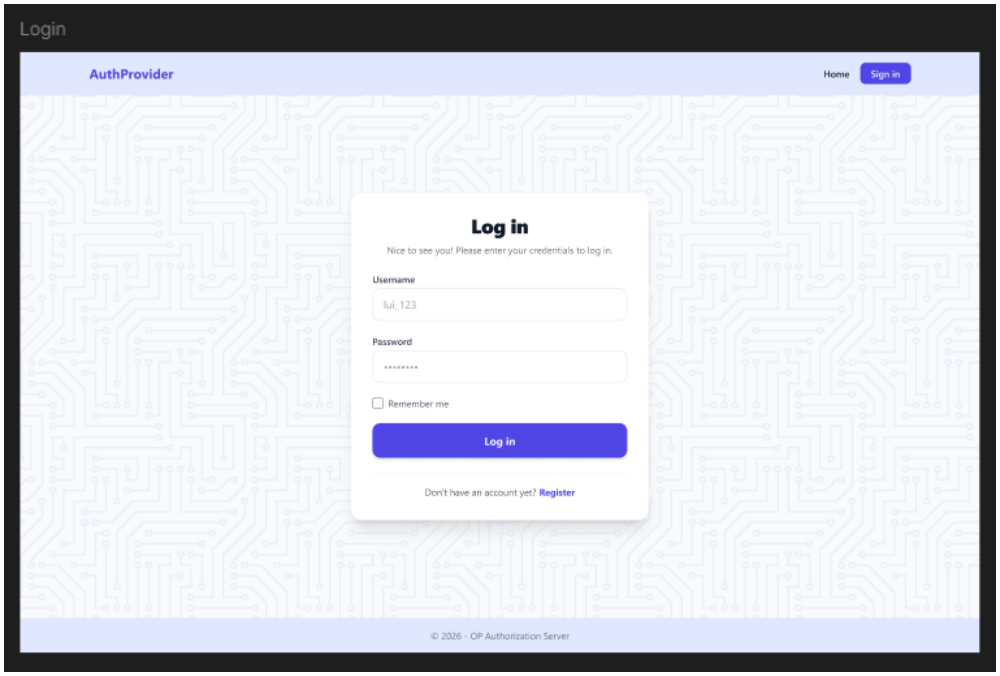


Рис. А.1. Прототип сторінки логіну

Джерело: розроблено автором.

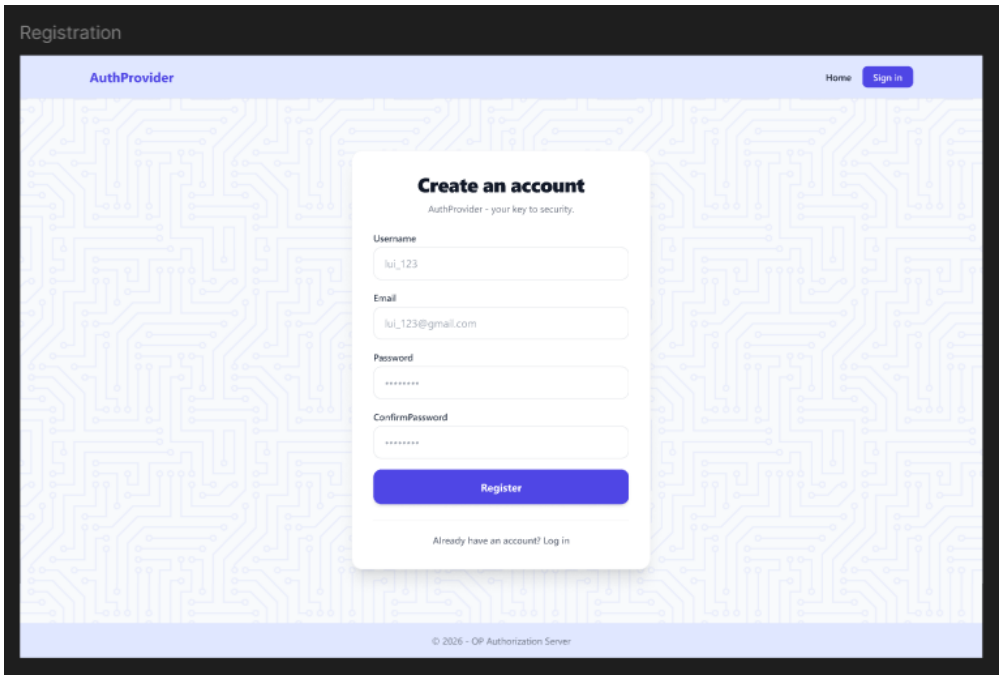


Рис. А.2. Прототип сторінки реєстрації

Джерело: розроблено автором.

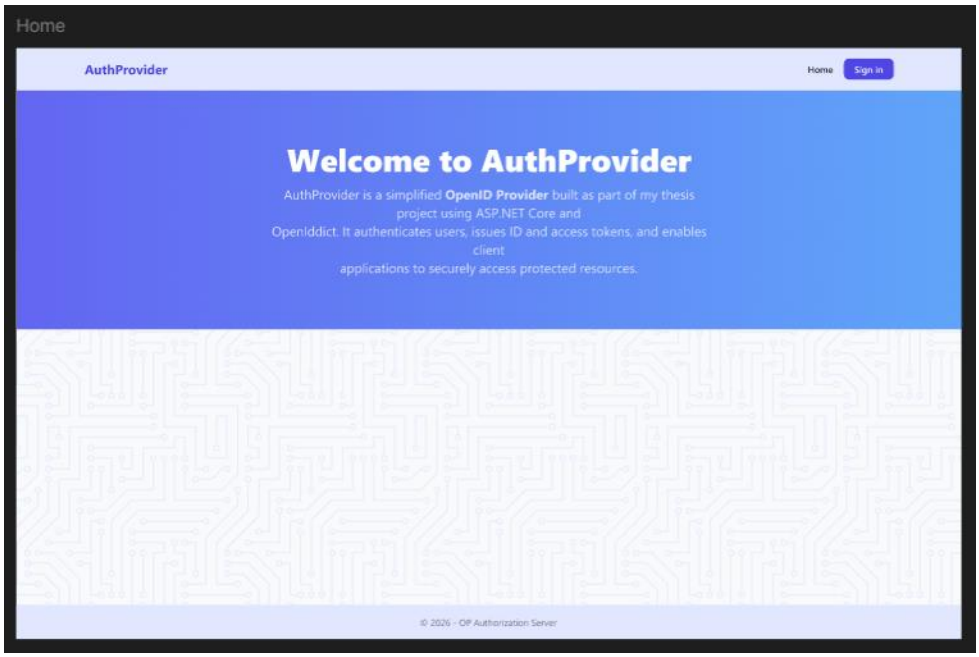
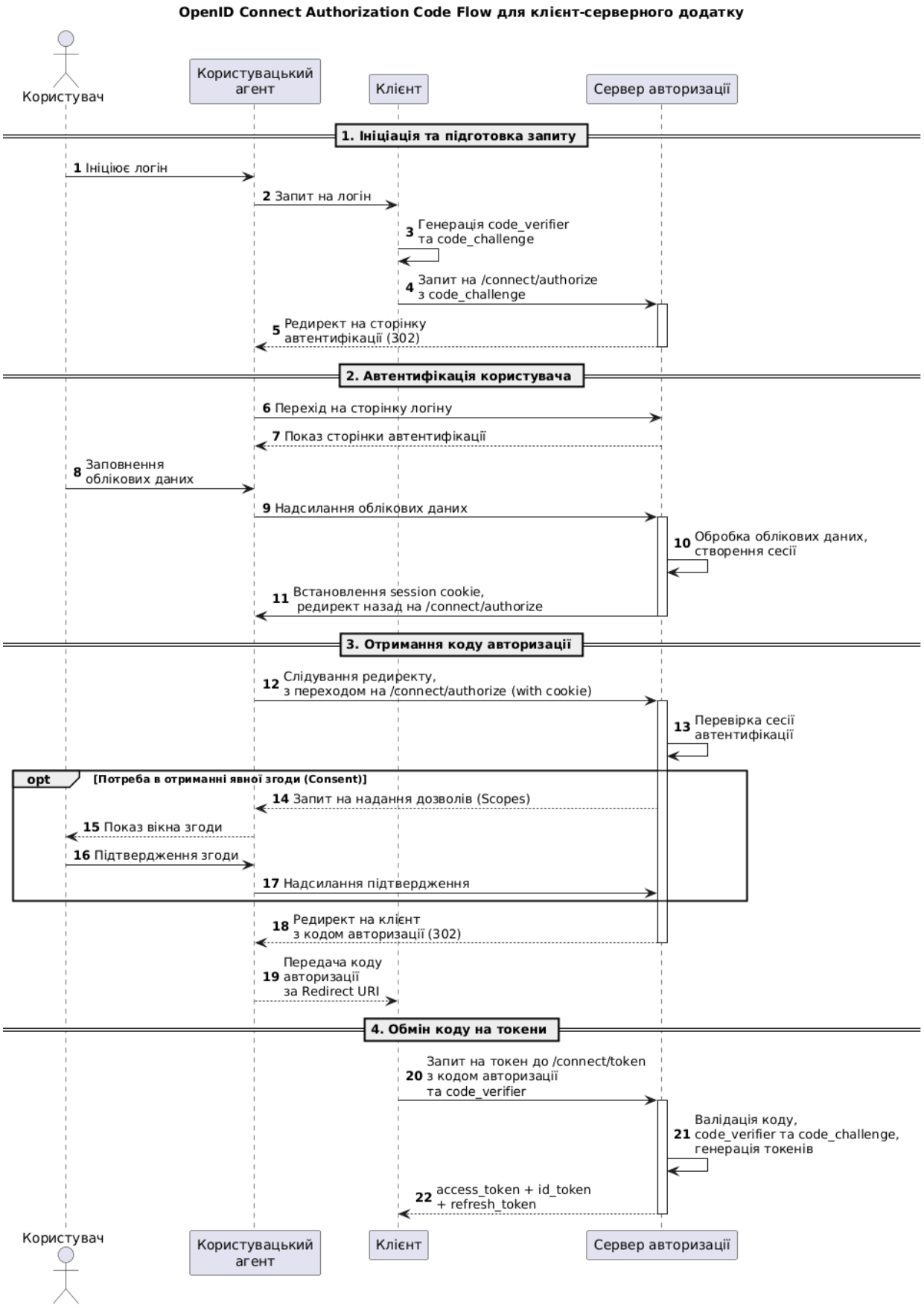


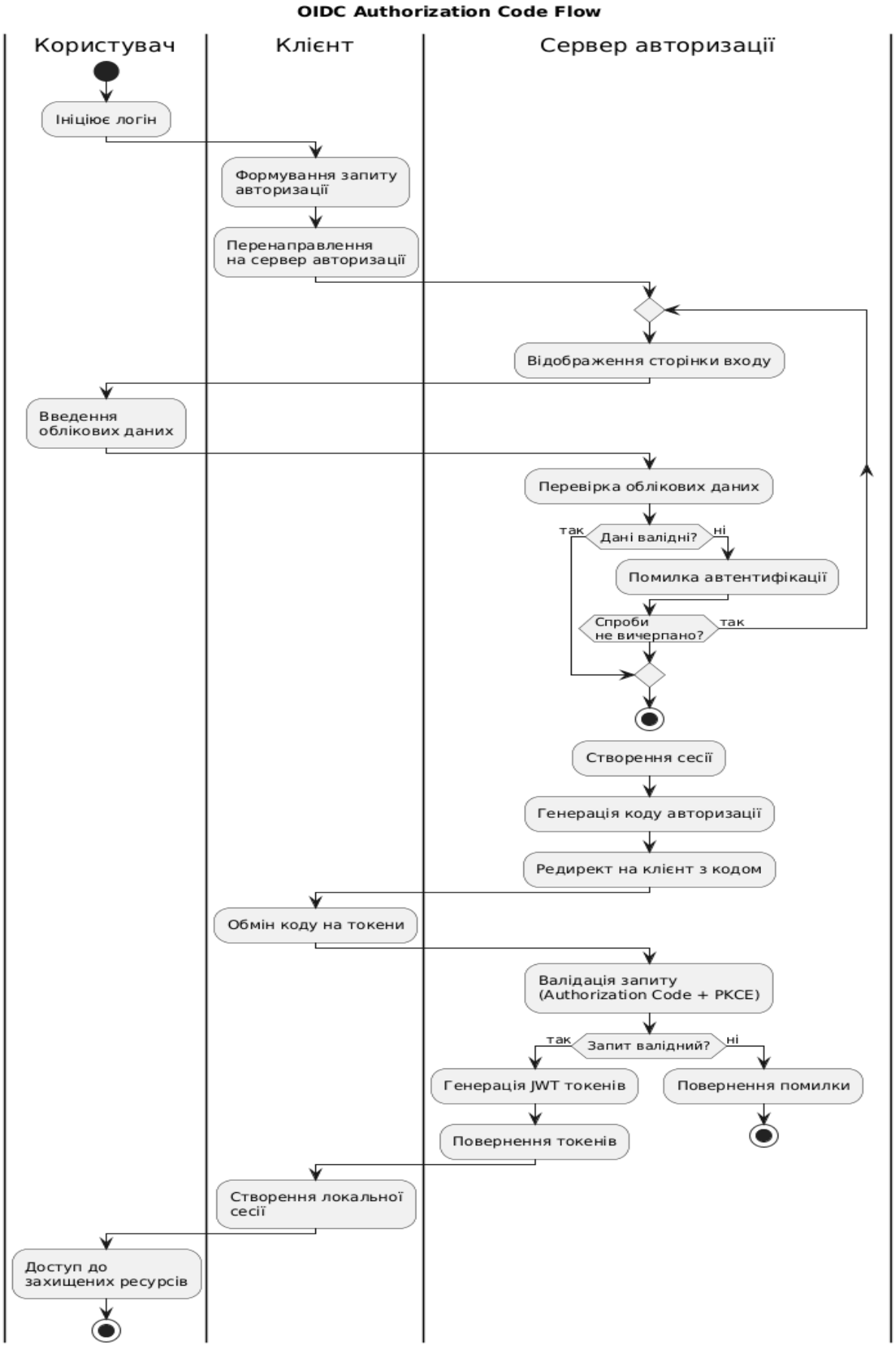
Рис. А.3. Прототип домашньої сторінки

Джерело: розроблено автором.

Додаток В


OpenID Connect Authorization Code Flow для клієнт-серверного додатку
Користувач
Користувацький агент
Клієнт
Сервер авторизації
1. Ініціація та підготовка запиту
1 Ініціює логін
2 Запит на логін
3 Генерація code_verifier та code_challenge
4 Запит на /connect/authorize з code_challenge
5 Редирект на сторінку автентифікації (302)
2. Автентифікація користувача
6 Перехід на сторінку логіну
7 Показ сторінки автентифікації
8 Заповнення облікових даних
9 Надсилання облікових даних
10 Обробка облікових даних, створення сесії
11 Встановлення session cookie, редирект назад на /connect/authorize
3. Отримання коду авторизації
12 Слідування редиректу, з переходом на /connect/authorize (with cookie)
13 Перевірка сесії автентифікації
opt
[Потреба в отриманні явної згоди (Consent)]
14 Запит на надання дозволів (Scopes)
15 Показ вікна згоди
16 Підтвердження згоди
17 Надсилання підтвердження
18 Редирект на клієнт з кодом авторизації (302)
19 Передача коду авторизації за Redirect URI
4. Обмін коду на токени
20 Запит на токен до /connect/token з кодом авторизації та code_verifier
21 Валідація коду, code_verifier та code_challenge, генерація токенів
22 access_token + id_token + refresh_token
Користувач
Користувацький агент
Клієнт
Сервер авторизації

Додаток Г


OIDC Authorization Code Flow
Користувач
Клієнт
Сервер авторизації
Ініціює логін
Формування запиту авторизації
Перенаправлення на сервер авторизації
Відображення сторінки входу
Введення облікових даних
Перевірка облікових даних
так
Дані валідні?
ні
Помилка автентифікації
Спроби не вичерпано?
так
Створення сесії
Генерація коду авторизації
Редирект на клієнт з кодом
Обмін коду на токени
Валідація запиту (Authorization Code + PKCE)
так
Запит валідний?
ні
Генерація JWT токенів
Повернення помилки
Повернення токенів
Створення локальної сесії
Доступ до захищених ресурсів